Árbol binario de búsqueda

Buscar, insertar y borrar bajando por un solo lado

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Decidir si un árbol binario es un árbol binario de búsqueda mirando cada subárbol completo, y explicar por qué su inorden sale ordenado (OA2).
- Escribir buscar, insertar y borrar en forma recursiva a partir de tres preguntas: cuál es el caso base, a qué subárbol se baja y qué se hace con lo que devuelve la llamada (OA3).
- Pasar buscar e insertar a un ciclo y comparar el espacio de las dos versiones (OA1, OA3).

Objetivos de aprendizaje II

Al finalizar esta sesión, el estudiante podrá también

- Seguir a mano cada operación sobre el árbol, incluidos los tres casos de borrar (OA2).
- Dar el costo $O(h)$ de cada operación y explicar, con un experimento, por qué la altura depende del orden de inserción (OA1).
- Resolver con recursión ejercicios y problemas de juez sobre el árbol binario de búsqueda (OA5).

Contenido I

1 Repaso: árbol binario y recursión

2 Buscar rápido sin perder el orden

3 El árbol binario de búsqueda

4 Buscar

5 Insertar

6 Borrar

7 Lo que cuesta: la forma del árbol

8 Ejercicios con recursión

9 Para el juez

Contenido I

1 Repaso: árbol binario y recursión

2 Buscar rápido sin perder el orden

3 El árbol binario de búsqueda

4 Buscar

5 Insertar

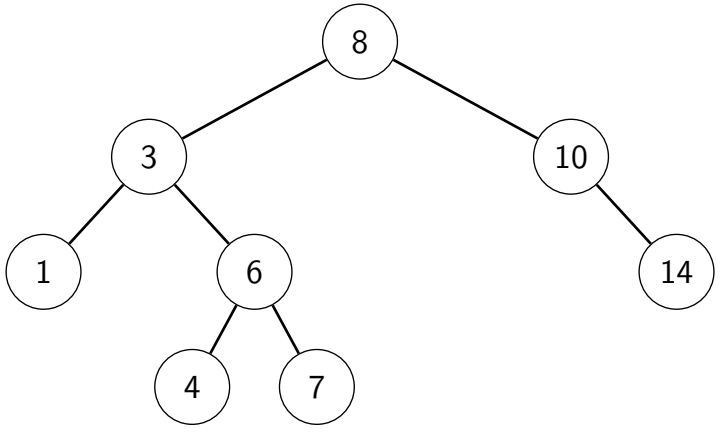
6 Borrar

7 Lo que cuesta: la forma del árbol

8 Ejercicios con recursión

9 Para el juez

Lo que quedó del árbol binario I



- Un árbol binario es vacío (`NULL`) o es un nodo con un dato, un subárbol izquierdo y un subárbol derecho.
- Preorden: `8 3 1 6 4 7 10 14`.
- Inorden: `1 3 4 6 7 8 10 14`.
- Posorden: `1 4 7 6 3 14 10 8`.
- Los tres recorridos cuestan $\Theta(n)$ y piden $O(h)$ de espacio aparte, en marcos de la pila de llamadas o en la pila explícita de `inordenPila`.

Pregunta

El inorden de este árbol sale de menor a mayor. ¿Casualidad, o hay algo en la manera en que están puestas las claves?

Caso base y caso recursivo I

La definición recursiva del árbol da funciones recursivas. Así quedó `tamano`:

```
int tamano(Nodo *r) {
  int res = 0;
  if (r != NULL) {
    res = 1 + tamano(r->izq) + tamano(r->der);
  }
  return res;
}
```

Cada función recursiva sobre un árbol responde tres preguntas.

Función	Caso base	¿A qué subárbol baja?	¿Qué hace con lo que vuelve?
`tamano`	vacío: 0	a los dos	1+ la suma
`altura`	vacío: -1	a los dos	1+ el mayor
`inorden`	vacío: nada	a los dos	nada; imprime en medio

Las tres bajan a los dos subárboles, por eso visitan los n nodos. Hoy aparece un árbol donde casi siempre basta con bajar a uno.

Contenido I

1 Repaso: árbol binario y recursión

2 Buscar rápido sin perder el orden

3 El árbol binario de búsqueda

4 Buscar

5 Insertar

6 Borrar

7 Lo que cuesta: la forma del árbol

8 Ejercicios con recursión

9 Para el juez

Un registro de carnés I

Problema

La biblioteca anota los carnés de quienes tienen un libro prestado. Llegan órdenes de cuatro tipos: agregar un carné, preguntar si un carné está, quitar un carné e imprimir todos de menor a mayor. Hay decenas de miles de carnés y las órdenes llegan en cualquier orden.

¿Entendimos el problema? I

Entradas

Una secuencia de órdenes: `agregar k`, `esta k`, `quitar k` y `listar`.

Salidas

`si` o `no` por cada `esta`, y los carnés en orden creciente por cada `listar`.

Orden	Salida
`agregar 8`, `agregar 3`, `agregar 10`, `esta 3`	`si`
`quitar 3`, `esta 3`	`no`
`listar`	`8 10`

Pregunta

¿Qué cuesta cada orden con una lista enlazada ordenada? ¿Y con un arreglo ordenado y búsqueda binaria?

¿Entendimos el problema? II

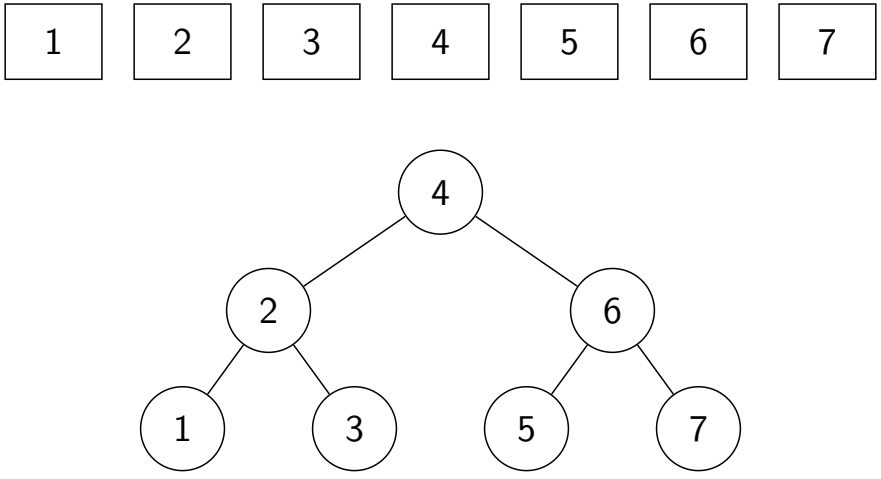
Respuesta

- Lista enlazada ordenada: enlazar cuesta $\Theta(1)$, pero antes hay que caminar hasta el lugar, así que agregar, buscar y quitar cuestan $\Theta(n)$. Listar es un recorrido, $\Theta(n)$.
- Arreglo ordenado: la búsqueda binaria encuentra en $\Theta(\log n)$, pero agregar y quitar corren los elementos que siguen, $\Theta(n)$.

	Buscar	Agregar	Quitar	Listar en orden
Lista ordenada	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Arreglo ordenado	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Tabla hash	$\Theta(1+\alpha)$ prom.	$\Theta(1+\alpha)$ prom.	$\Theta(1+\alpha)$ prom.	ordenar: $\Theta(n \log n)$
Lo que se quiere	$O(\log n)$	$O(\log n)$	$O(\log n)$	$\Theta(n)$

La búsqueda binaria ya dibuja un árbol I

Sobre el arreglo 1 2 3 4 5 6 7, la búsqueda binaria mira primero el 4; después el 2 o el 6; después uno de 1, 3, 5 o 7. Las comparaciones posibles forman un árbol:



Pregunta

Si cada casilla del medio fuera un nodo con punteros a sus dos mitades, ¿qué habría que correr para agregar el 8?

La búsqueda binaria ya dibuja un árbol II

Respuesta

Nada. El 8 se compara con 4, con 6 y con 7, y se cuelga como hijo derecho del 7. Con nodos y punteros, insertar es seguir el camino de la búsqueda y enlazar un nodo nuevo al final. Esa estructura es el árbol binario de búsqueda.

Contenido I

1 Repaso: árbol binario y recursión

2 Buscar rápido sin perder el orden

3 El árbol binario de búsqueda

4 Buscar

5 Insertar

6 Borrar

7 Lo que cuesta: la forma del árbol

8 Ejercicios con recursión

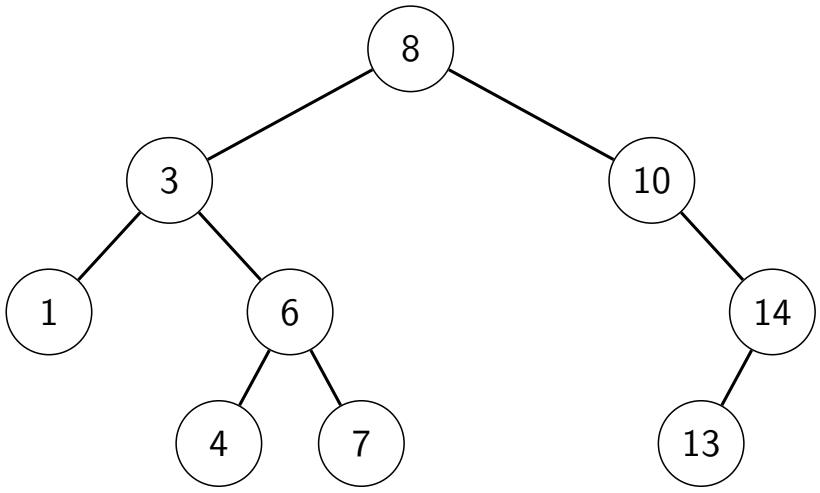
9 Para el juez

Definición (CLRS, §12.1; Thareja, §10.1) I

Árbol binario de búsqueda (ABB)

Es un árbol binario donde, para cada nodo x, toda clave del subárbol izquierdo de x es menor que la clave de x, y toda clave del subárbol derecho es mayor.

CLRS admite claves repetidas con $\leq$ y $\geq$. Aquí las claves son distintas: insertar una clave que ya está deja el árbol igual.

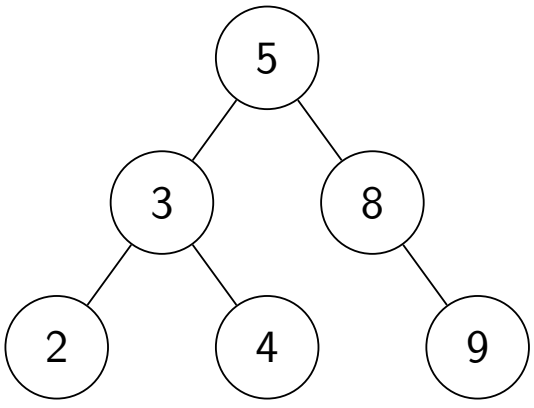


Ejemplo

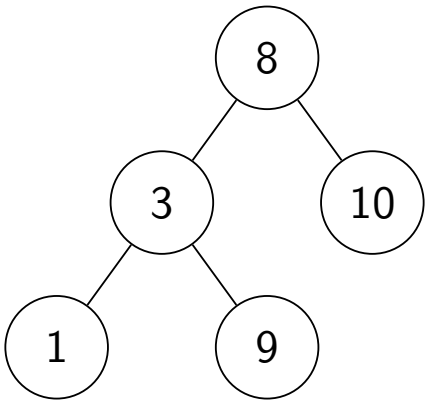
En el 8, a la izquierda quedan 1, 3, 4, 6 y 7, todos menores; a la derecha, 10, 13 y 14, todos mayores. En el 3, a la izquierda está el 1 y a la derecha 4, 6 y 7. Lo mismo pasa en cada nodo.

¿Cuáles son ABB? I

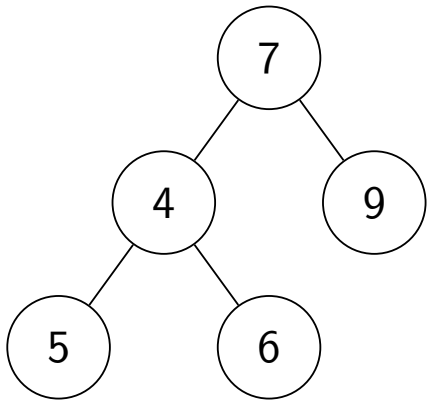
A



B



C



Pregunta

Un compañero revisa cada nodo contra sus dos hijos: el izquierdo debe ser menor y el derecho mayor. ¿Qué responde para A, B y C? ¿Acierta?

¿Cuáles son ABB? II

Respuesta

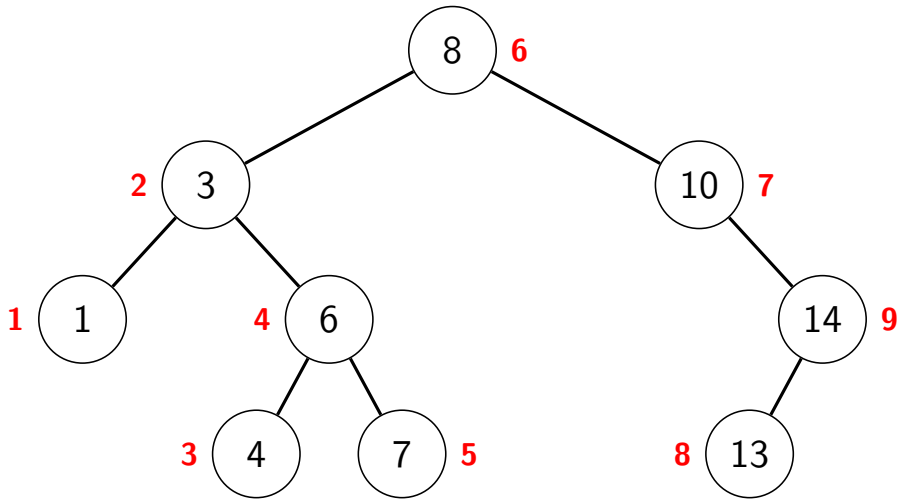
- A es ABB, y el compañero acierta.
- C no es ABB: el 5 es hijo izquierdo del 4. El compañero también lo detecta.
- B engaña al compañero. Cada nodo cumple con sus hijos (el 9 es mayor que su padre 3), pero el 9 está en el subárbol izquierdo del 8 y es mayor que 8. B no es ABB.

El error de mirar solo a los hijos

La definición habla de todo el subárbol, no de los hijos inmediatos. En los ejercicios aparece la función que lo revisa bien, con el rango de valores que le toca a cada nodo.

El inorden de un ABB sale ordenado I

El inorden imprime primero todo el subárbol izquierdo, después la raíz y después todo el subárbol derecho. En un ABB eso es: primero las claves menores que la raíz, luego la raíz, luego las mayores. Dentro de cada subárbol pasa lo mismo, así que nadie sale fuera de su lugar.



En rojo, el turno de cada nodo en el inorden: `1 3 4 6 7 8 10 13 14`. Esto responde la pregunta del repaso: el árbol del repaso era un ABB. El contenedor `map` recorre sus claves en orden por la misma razón: por dentro guarda un árbol de búsqueda.

El ABB en C++ I

El nodo es el del árbol binario: un dato y dos punteros.

```
struct Nodo {
  int dato;
  Nodo *izq;  // raiz del subarbol izquierdo
  Nodo *der;  // raiz del subarbol derecho
};
```

abb.h incluye arbol_binario.h y le agrega las operaciones de búsqueda:

```
// Arbol binario de busqueda de enteros sin repetidos: en cada nodo,
↪  las
// claves del subarbol izquierdo son menores y las del derecho,
↪  mayores.
// Insertar una clave que ya esta no cambia el arbol.
#ifndef ABB_H
#define ABB_H
#include <cassert>
#include "arbol_binario.h"
```

El ABB en C++ II

Estas son las funciones que se construyen en lo que sigue:

Función	Devuelve
`buscar(r, k)`, `buscarIterativa(r, k)`	el nodo con la clave `k`, o `NULL`
`minimo(r)`, `maximo(r)`	el nodo con la menor o la mayor clave
`insertar(r, k)`, `insertarIterativa(r, k)`	la raíz del árbol con `k` adentro
`borrar(r, k)`	la raíz del árbol sin `k`

Contenido I

1 Repaso: árbol binario y recursión

2 Buscar rápido sin perder el orden

3 El árbol binario de búsqueda

4 Buscar

5 Insertar

6 Borrar

7 Lo que cuesta: la forma del árbol

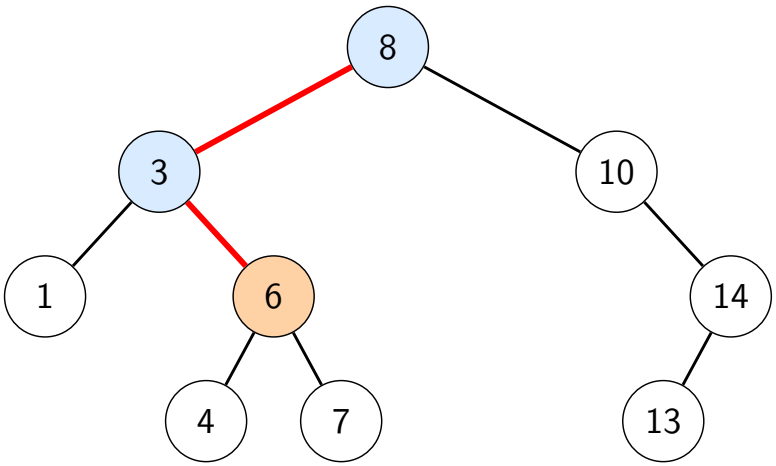
8 Ejercicios con recursión

9 Para el juez

Buscar 6 y buscar 5 a mano I

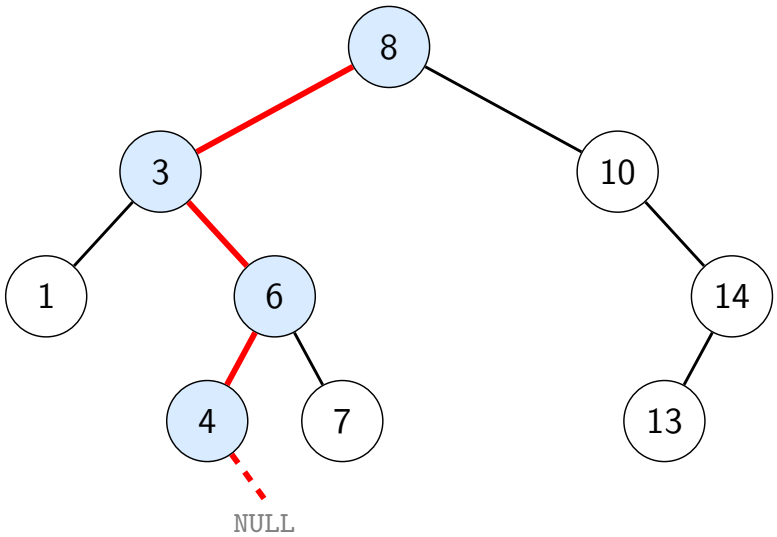
En cada nodo se compara la clave buscada con el dato y se baja a un solo lado: a la izquierda si es menor, a la derecha si es mayor.

buscar 6: $6 < 8$, $6 > 3$, $6 = 6$



Se encuentra en el tercer nodo.

buscar 5: $5 < 8$, $5 > 3$, $5 < 6$, $5 > 4$



El hijo derecho de 4 es `NULL`: el 5 no está.

Las tres preguntas para buscar I

Pregunta

Para escribir `buscar(r, k)` de forma recursiva:

1. ¿Cuál es el caso base, el que se responde sin llamar a nadie?
2. ¿A qué subárbol se baja?
3. ¿Qué se hace con lo que devuelve la llamada?

Las tres preguntas para buscar II

Respuesta

1. Hay dos casos base. Si `r` es `NULL`, la clave no está y se devuelve `NULL`. Si `r->dato` es `k`, se devuelve `r`.
2. A uno solo: al izquierdo si `k < r->dato`, al derecho si no. El otro subárbol no puede tener a `k`.
3. Nada: lo que encuentra el subárbol es la respuesta de todo el árbol, y se devuelve tal cual.

buscar en C++ (CLRS, §12.2) I

```
// Devuelve el nodo con la clave k, o NULL si no esta.
Nodo *buscar(Nodo *r, int k) {
  Nodo *res = r;
  if (r != NULL && k != r->dato) {
    if (k < r->dato) {
      res = buscar(r->izq, k);
    } else {
      res = buscar(r->der, k);
    }
  }
  return res;
}
```

- `res = r` cubre los dos casos base de una vez: si `r` es `NULL` se devuelve `NULL`, y si tiene la clave se devuelve el mismo `r`.
- Solo una de las dos llamadas se ejecuta en cada nodo. Ahí está la diferencia con `tamano`, que baja a los dos lados.

La pila de llamadas de `buscar(raiz, 5)` I

Marco	`r->dato`	Comparación	Hace
1	8	$5 < 8$	`res = buscar(r->izq, 5)`
2	3	$5 > 3$	`res = buscar(r->der, 5)`
3	6	$5 < 6$	`res = buscar(r->izq, 5)`
4	4	$5 > 4$	`res = buscar(r->der, 5)`
5	`NULL`	caso base	devuelve `NULL`

En el momento más hondo hay cinco marcos vivos, uno por nivel recorrido. Después el `NULL` sube: el marco 4 lo guarda en `res` y lo devuelve, el 3 hace lo mismo, y así hasta el primero. Ningún marco cambia lo que recibe.

Pregunta

Si la llamada recursiva es lo último que hace cada marco y su resultado no se toca, ¿hace falta guardar los marcos de arriba?

La pila de llamadas de `buscar(raiz, 5)` II

Respuesta

No. Cada marco solo espera para devolver lo mismo que le llegue. Se puede cambiar la llamada `buscar(r->izq, k)` por la asignación `r = r->izq` y repetir en un ciclo: la búsqueda baja igual y no apila nada.

De la recursión al ciclo I

```
// La misma busqueda con un ciclo: baja un nivel por vuelta.
Nodo *buscarIterativa(Nodo *r, int k) {
  while (r != NULL && k != r->dato) {
    if (k < r->dato) {
      r = r->izq;
    } else {
      r = r->der;
    }
  }
  return r;
}
```

De la recursión al ciclo II

El ciclo sigue mientras quede árbol y no se haya encontrado la clave. La condición del `while` es la negación de los dos casos base.

`buscarIterativa(raiz, 13)`, vuelta por vuelta

Vuelta	`r`	Comparación
antes del ciclo	8	$13 > 8$: `r = r->der`
1	10	$13 > 10$: `r = r->der`
2	14	$13 < 14$: `r = r->izq`
3	13	igual: el ciclo termina

Devuelve el nodo del 13. En `uso_abb.cpp` las dos versiones responden lo mismo para 6, 13, 5 y 15.

Mínimo y máximo (CLRS, §12.2) I

Pregunta

¿Dónde está la menor clave de un ABB? ¿Es siempre una hoja?

Mínimo y máximo (CLRS, §12.2) II

Respuesta

Al fondo de la izquierda: todo lo que está a la izquierda de un nodo es menor que él, así que se baja por `izq` mientras se pueda. No siempre es una hoja: si se inserta el 2 en el árbol de los ejemplos, queda como hijo derecho del 1, y el 1 sigue siendo el mínimo.

Mínimo y máximo (CLRS, §12.2) III

```
// exige r != NULL; el menor esta al fondo a la izquierda
Nodo *minimo(Nodo *r) {
  assert(r != NULL);
  while (r->izq != NULL) {
    r = r->izq;
  }
  return r;
}

// exige r != NULL; el mayor esta al fondo a la derecha
Nodo *maximo(Nodo *r) {
  assert(r != NULL);
  while (r->der != NULL) {
    r = r->der;
  }
  return r;
}
```

Mínimo y máximo (CLRS, §12.2) IV

En el árbol de los ejemplos, `minimo` pasa por 8, 3 y 1, y `maximo` por 8, 10 y 14. La salida de `uso_abb.cpp` lo confirma: `minimo: 1, maximo: 14`.

Caso límite

En un árbol vacío no hay mínimo. Las dos funciones exigen `r != NULL` y lo revisan con `assert`: llamarlas sobre `NULL` detiene el programa en vez de desreferenciar un puntero nulo.

Lo que cuesta buscar I

Sea h la altura del árbol. Cada vuelta o cada llamada baja un nivel, y un camino desde la raíz pasa por a lo sumo $h + 1$ nodos.

Función	Tiempo	Espacio aparte	Por qué
`buscar`	$O(h)$	$O(h)$	una comparación por nivel; un marco por cada nivel que baja
`buscarIterativa`	$O(h)$	$O(1)$	el mismo camino, con una sola variable `r`
`minimo`, `maximo`	$O(h)$	$O(1)$	bajan por un lado hasta el fondo

Cuánto vale h depende de la forma del árbol, y eso se mide más adelante. Por ahora basta ver que ninguna de las tres recorre el árbol entero.

Contenido I

1 Repaso: árbol binario y recursión

2 Buscar rápido sin perder el orden

3 El árbol binario de búsqueda

4 Buscar

5 Insertar

6 Borrar

7 Lo que cuesta: la forma del árbol

8 Ejercicios con recursión

9 Para el juez

¿Dónde va el 5? I

Pregunta

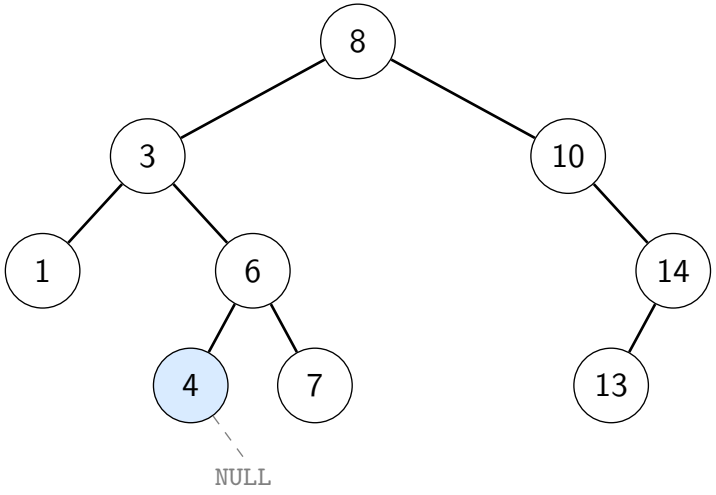
Se quiere insertar el 5 en el árbol de los ejemplos sin mover ningún nodo que ya esté. ¿Dónde queda?

¿Dónde va el 5? II

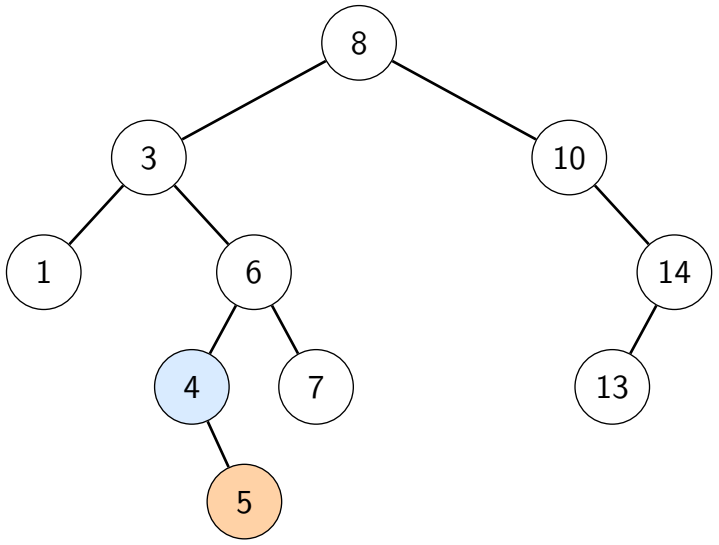
Respuesta

Donde la búsqueda del 5 se cae: el hijo derecho del 4, que era `NULL`. Allí se cuelga un nodo nuevo y nada más cambia.

antes



después



Las tres preguntas para insertar I

Pregunta

`insertar(r, k)` devuelve la raíz del subárbol que queda con `k` adentro. ¿Cuál es el caso base? ¿A qué subárbol se baja? ¿Qué se hace con lo que devuelve la llamada?

Las tres preguntas para insertar II

Respuesta

1. Caso base: `r` es `NULL`. Se crea el nodo de `k`, y ese nodo es la raíz del subárbol nuevo.
2. Se baja como en `buscar`: al izquierdo si `k < r->dato`, al derecho si `k > r->dato`. Si `k` ya está, no se baja.
3. Lo que devuelve la llamada se guarda en el mismo hijo por el que se bajó, y se devuelve `r`.

insertar en C++ (CLRS, §12.3) I

```
// Devuelve la raiz del subarbol con k ya insertada.
Nodo *insertar(Nodo *r, int k) {
  if (r == NULL) {
    r = crearNodo(k, NULL, NULL);
  } else if (k < r->dato) {
    r->izq = insertar(r->izq, k);
  } else if (k > r->dato) {
    r->der = insertar(r->der, k);
  }
  return r;
}
```

- Si `k == r->dato`, ninguna rama se ejecuta y se devuelve `r` sin cambios: un repetido se ignora.
- La primera llamada también asigna: `raiz = insertar(raiz, k)`. Con el árbol vacío, esa es la única forma de que `raiz` deje de ser `NULL`.

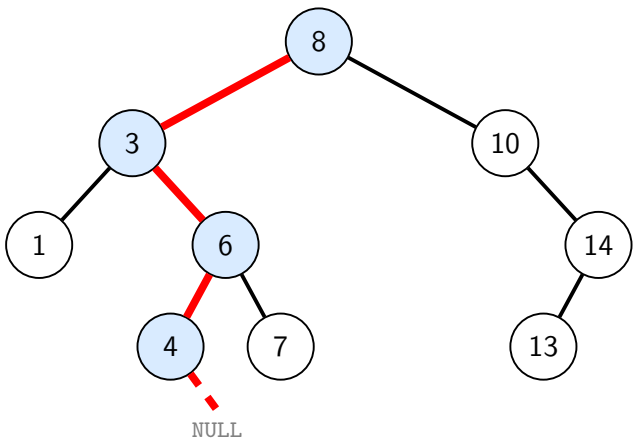
`insertar` en C++ (CLRS, §12.3) II

Pregunta

`r->izq = insertar(r->izq, k)` reescribe un puntero en cada nodo del camino. ¿Cuál de esas asignaciones cambia algo de verdad?

Por qué `r->izq = insertar(r->izq, k)` reengancha el árbol I

Al bajar, cada marco queda esperando para asignar lo que le devuelvan.

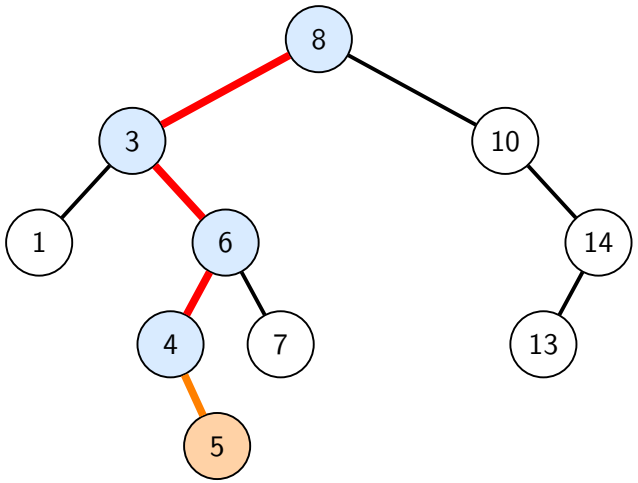


pila de llamadas (tope arriba)

`insertar(NULL, 5)`
`insertar(nodo 4, 5): r->der = ?`
`insertar(nodo 6, 5): r->izq = ?`
`insertar(nodo 3, 5): r->der = ?`
`insertar(nodo 8, 5): r->izq = ?`
`main: raiz = ?`

Por qué `r->izq = insertar(r->izq, k)` reengancha el árbol II

El caso base crea el 5 y devuelve su dirección. El marco del 4 la guarda en `r->der`: esta es la asignación que engancha el nodo.

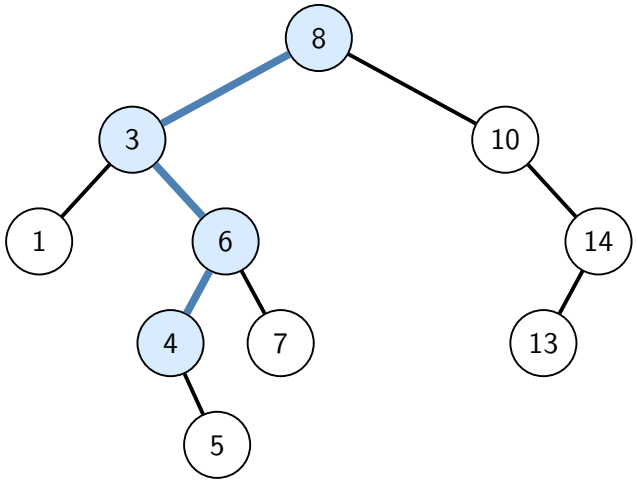


pila de llamadas (tope arriba)

`insertar(NULL, 5): crea el 5`
`insertar(nodo 4, 5): r->der = nodo 5`
`insertar(nodo 6, 5): r->izq = ?`
`insertar(nodo 3, 5): r->der = ?`
`insertar(nodo 8, 5): r->izq = ?`
`main: raiz = ?`

Por qué `r->izq = insertar(r->izq, k)` reengancha el árbol III

Al subir, cada marco devuelve su `r` y el de arriba lo guarda en el mismo hijo por el que bajó. Esos punteros ya valían eso: se reescriben con el mismo valor.



asignaciones al subir

```
nodo 4 devuelve nodo 4
insertar(nodo 6, 5): r->izq = nodo 4
insertar(nodo 3, 5): r->der = nodo 6
insertar(nodo 8, 5): r->izq = nodo 3
main: raiz = nodo 8
```

Por qué `r->izq = insertar(r->izq, k)` reengancha el árbol IV

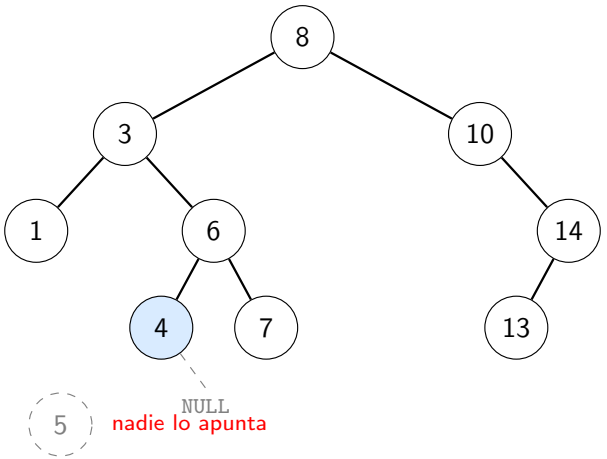
Respuesta

Solo cambia el puntero donde estaba el `NULL`: el `r->der` del 4. Las demás asignaciones ponen el valor que ya había. El mismo código sirve para el árbol vacío: en `raiz = insertar(NULL, 8)` el caso base devuelve el nodo del 8 y la asignación que cambia es la de `raiz`. Así no hace falta preguntar por el padre ni tratar aparte la raíz.

Si la llamada no se asigna I

Error común

Si la línea se escribe `insertar(r->der, k);`, sin `r->der =` delante, el nodo se crea igual, pero nadie guarda su dirección. El parámetro `r` de la llamada es una copia del puntero: asignarle el nodo nuevo no toca el `r->der` del padre.

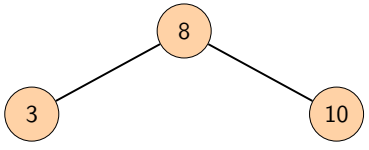


El 5 no aparece en el árbol y su memoria queda reservada sin forma de liberarla: una fuga.

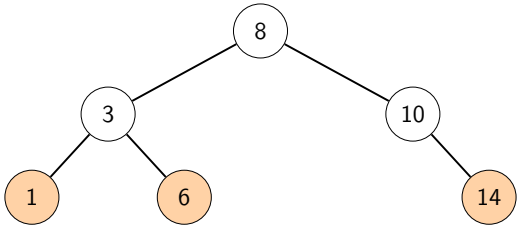
El árbol de los ejemplos, clave por clave I

`uso_abb.cpp` inserta `8 3 10 1 6 14 4 7 13` en un árbol vacío. En naranja, las claves que entran en cada tramo.

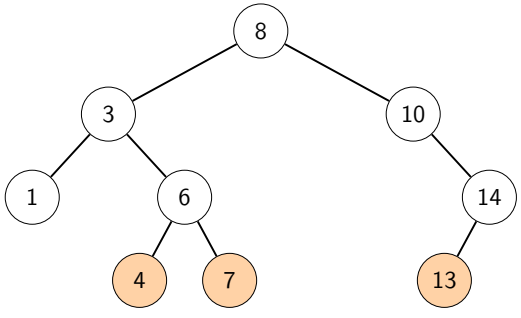
8, 3, 10



1, 6, 14



4, 7, 13



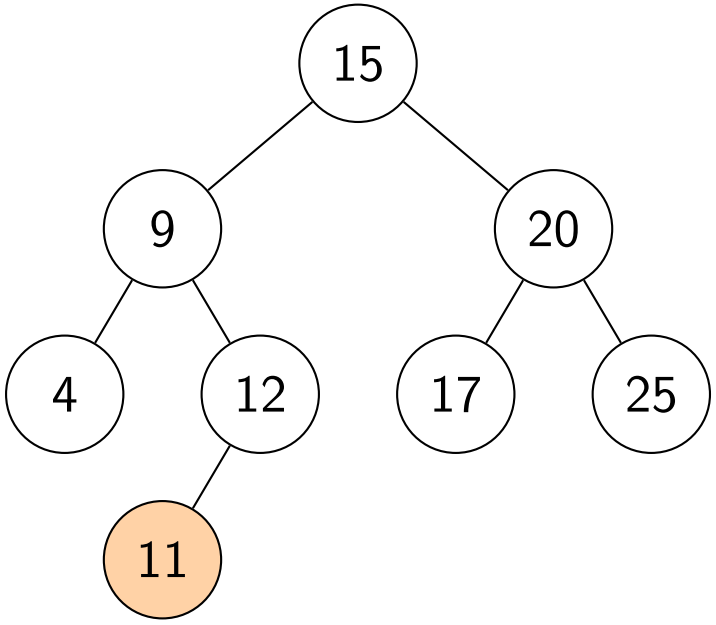
El 13 hace el camino más largo: $13 > 8$, $13 > 10$, $13 < 14$, y entra como hijo izquierdo del 14. Después el programa inserta otra vez el 6, y el árbol no cambia. El preorden que imprime es `8 3 1 6 4 7 10 14 13`.

Ejercicio en parejas: otro árbol I

Las preguntas

1. Inserten `15 9 20 4 12 17 25 11`, en ese orden, en un ABB vacío, y dibujen el árbol.
2. Escriban su preorden y su inorden. ¿Cuál es su altura?
3. ¿Qué marcos hay en la pila de llamadas en el momento en que se crea el nodo del 11?

Ejercicio en parejas: otro árbol II



Lo que debe salir

- Preorden: `15 9 4 12 11 20 17 25`.
- Inorden: `4 9 11 12 15 17 20 25`, ordenado.
- Altura 3, por el camino 15, 9, 12, 11.
- Al crear el 11 hay cinco marcos: los de 15, 9 y 12, y el de `insertar(NULL, 11)` encima, con el de `main` debajo de todos.

`insertarIterativa` (CLRS, §12.3) I

Sin recursión no hay quién reenganche al subir, así que el ciclo recuerda al `padre` del nodo actual y al final cuelga ahí el nodo nuevo. Si `padre` queda en `NULL`, el árbol estaba vacío y el nodo nuevo es la raíz.

insertarIterativa (CLRS, §12.3) II

```
Nodo *insertarIterativa(Nodo *r, int k) {
  Nodo *padre = NULL;
  Nodo *actual = r;
  while (actual != NULL && k != actual->dato) {
    padre = actual;
    if (k < actual->dato) {
      actual = actual->izq;
    } else {
      actual = actual->der;
    }
  }
  if (actual == NULL) {
    Nodo *nuevo = crearNodo(k, NULL, NULL);
    if (padre == NULL) {
      r = nuevo;
    } else if (k < padre->dato) {
      padre->izq = nuevo;
    } else {
      padre->der = nuevo;
    }
  }
  return r;
}
```

`insertarIterativa(raiz, 5)`, vuelta por vuelta I

Momento	`padre`	`actual`	Comparación
antes del ciclo	`NULL`	8	$5 < 8$: a la izquierda
después de la vuelta 1	8	3	$5 > 3$: a la derecha
después de la vuelta 2	3	6	$5 < 6$: a la izquierda
después de la vuelta 3	6	4	$5 > 4$: a la derecha
después de la vuelta 4	4	`NULL`	el ciclo termina

`actual` quedó en `NULL`: el 5 no estaba. Se crea el nodo, y como $5 > 4$ se cuelga en `padre->der`. Es el mismo árbol que deja la versión recursiva.

`insertarIterativa(raiz, 5)`, vuelta por vuelta II

Casos límite

- Árbol vacío: el ciclo no da ninguna vuelta, `padre` sigue en `NULL` y el nodo nuevo pasa a ser la raíz.
- Clave repetida: el ciclo termina con `actual` en el nodo que la tiene y no se crea nada.

Lo que cuesta insertar I

Función	Tiempo	Espacio aparte	Por qué
`insertar`	$O(h)$	$O(h)$	baja un camino; un marco por nivel, que espera para reenganchar
`insertarIterativa`	$O(h)$	$O(1)$	el mismo camino con dos punteros, `padre` y `actual`

Ninguna de las dos corre nodos: enlazar el nuevo cuesta $\Theta(1)$. Esa es la ganancia frente al arreglo ordenado, que para insertar corre hasta n elementos.

Lo que cuesta insertar II

El programa de uso arma el árbol, busca con las dos versiones y pregunta por el mínimo y el máximo:

```
int main() {
  int claves[9] = {8, 3, 10, 1, 6, 14, 4, 7, 13};
  Nodo *raiz = NULL;
  int i = 0;
  while (i < 9) {
    raiz = insertar(raiz, claves[i]);
    i = i + 1;
  }
  raiz = insertar(raiz, 6);   // repetido: se ignora
  printf("tras insertar 8 3 10 1 6 14 4 7 13 (y 6 otra vez):\n");
  mostrar(raiz);
  printf("tamano: %d, altura: %d\n", tamano(raiz), altura(raiz));
  printf("minimo: %d, maximo: %d\n", minimo(raiz)->dato,
  ↪  maximo(raiz)->dato);

  probarBusqueda(raiz, 6);
  probarBusqueda(raiz, 13);
  probarBusqueda(raiz, 5);
  probarBusqueda(raiz, 15);
```

Lo que cuesta insertar III

Su salida, hasta las búsquedas:

```
tras insertar 8 3 10 1 6 14 4 7 13 (y 6 otra vez):
  inorden:  1 3 4 6 7 8 10 13 14
  preorden: 8 3 1 6 4 7 10 14 13
tamano: 9, altura: 3
minimo: 1, maximo: 14
buscar(6): esta, buscarIterativa(6): esta
buscar(13): esta, buscarIterativa(13): esta
buscar(5): no esta, buscarIterativa(5): no esta
buscar(15): no esta, buscarIterativa(15): no esta
```

Contenido I

1 Repaso: árbol binario y recursión

2 Buscar rápido sin perder el orden

3 El árbol binario de búsqueda

4 Buscar

5 Insertar

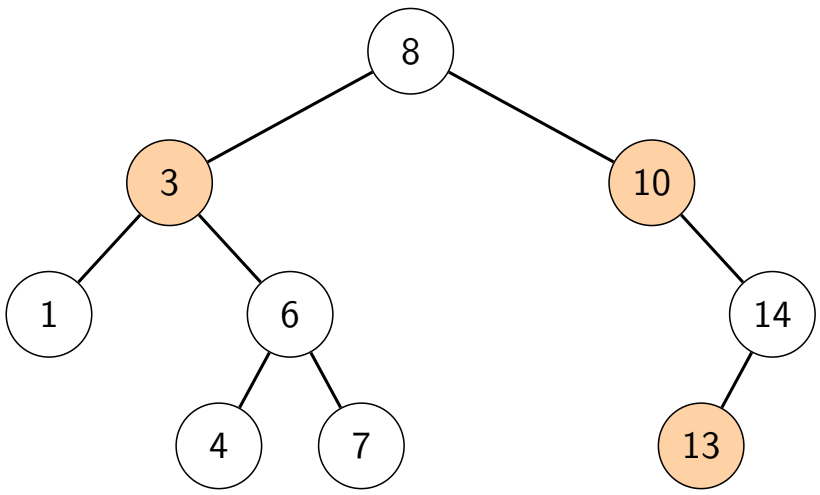
6 Borrar

7 Lo que cuesta: la forma del árbol

8 Ejercicios con recursión

9 Para el juez

Quitar un nodo sin romper el árbol I



Pregunta

`uso_abb.cpp` borra el 13, después el 10 y después el 3. ¿Qué tiene de distinto cada uno de esos nodos en el momento de borrarlo? ¿Qué le pasa a lo que cuelga de él?

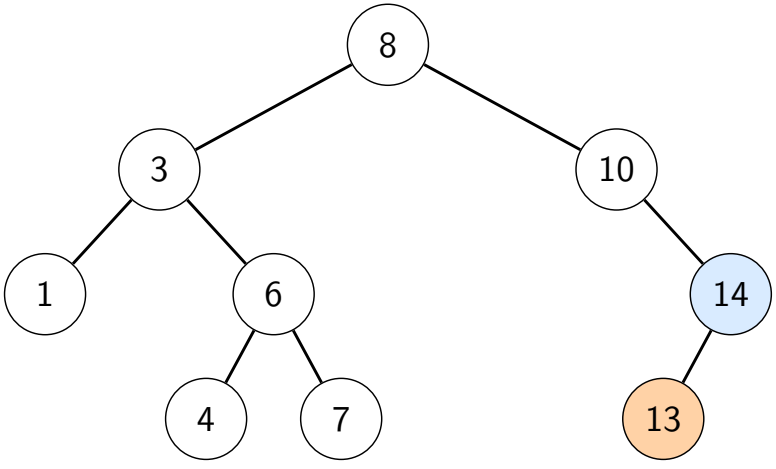
Quitar un nodo sin romper el árbol II

Respuesta (CLRS, §12.3; Thareja, §10.2)

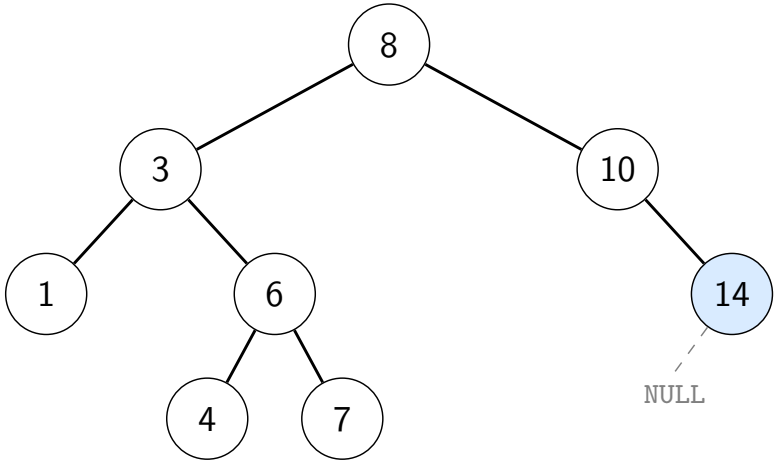
- El 13 es una hoja: se quita y su padre apunta a `NULL`.
- Cuando se borra el 10, ya sin el 13, tiene un solo hijo, el 14. El 14 puede subir a su lugar.
- El 3 tiene dos hijos, 1 y 6. Un solo puntero del 8 no alcanza para los dos: hay que poner otra clave en el lugar del 3.

Caso 1: una hoja (borrar 13) I

antes



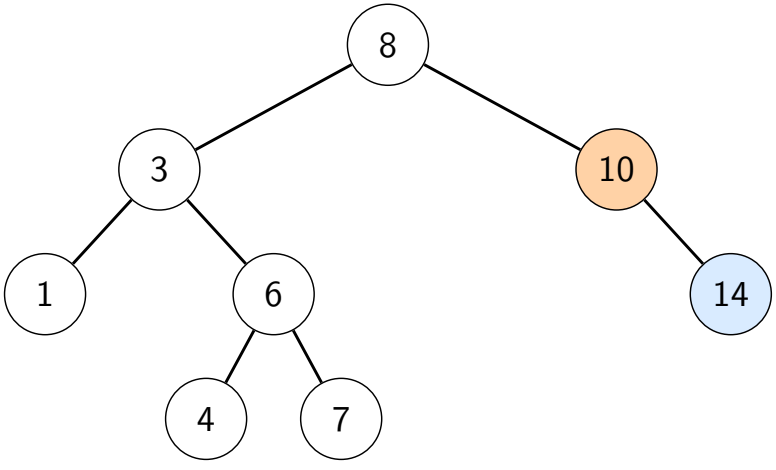
después



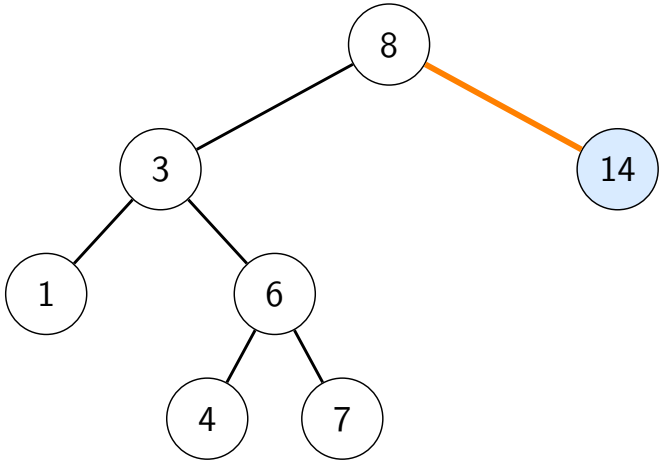
El hijo izquierdo del 14 pasa a ser `NULL` y el nodo se libera. Nadie más se mueve. El programa imprime después `inorden: 1 3 4 6 7 8 10 14` y `preorden: 8 3 1 6 4 7 10 14`.

Caso 2: un solo hijo (borrar 10) I

antes



después



El 14 sube al lugar del 10 con todo lo que cuelga de él: el 8 pasa a apuntar al 14. Sigue siendo ABB, porque todo lo que estaba debajo del 10 ya era mayor que 8. Salida: `inorden: 1 3 4 6 7 8 14`, `preorden: 8 3 1 6 4 7 14`.

Caso 3: dos hijos (borrar 3) I

Pregunta

¿Qué clave del árbol puede quedar en el lugar del 3 sin que deje de ser ABB? Tiene que ser mayor que todo lo que quede a la izquierda y menor que todo lo que quede a la derecha.

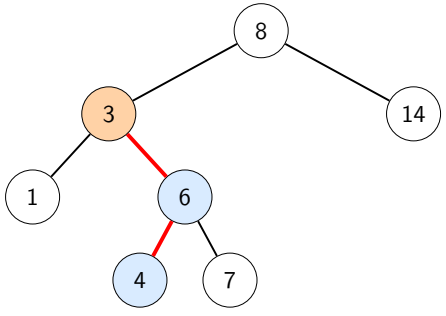
Caso 3: dos hijos (borrar 3) II

Respuesta

El sucesor del 3: la clave que le sigue en el inorden, que es el mínimo de su subárbol derecho. Aquí es el 4. Es mayor que el 1 y menor que 6 y 7. El sucesor nunca tiene hijo izquierdo (ese hijo sería menor que él), así que quitarlo de abajo es el caso 1 o el caso 2.

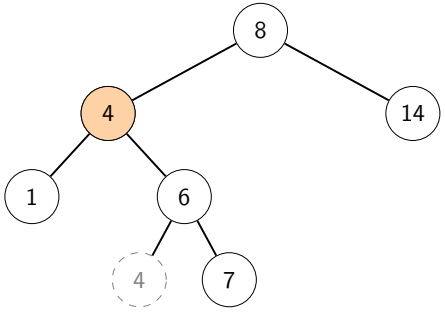
Caso 3: dos hijos (borrar 3) III

1. buscar el sucesor



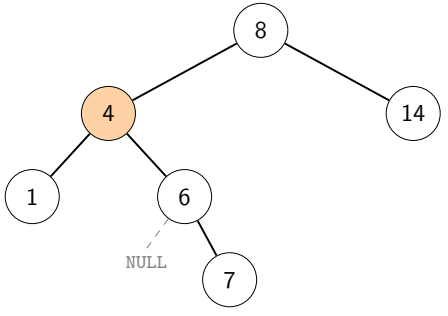
`minimo(r->der)`: 6, 4.

2. copiar su clave



`r->dato = 4`; el 4 de abajo sobra.

3. borrarlo de abajo



el 4 de abajo era hoja: caso 1.

Salida: `inorden: 1 4 6 7 8 14`, `preorden: 8 4 1 6 7 14`. El inorden no cambió de orden: solo perdió el 3.

Las tres preguntas para borrar I

Pregunta

`borrar(r, k)` devuelve la raíz del subárbol que queda sin `k`.
¿Cuál es el caso base? ¿A qué subárbol se baja? ¿Qué se hace con lo que devuelve la llamada?

Las tres preguntas para borrar II

Respuesta

1. Casos base: `r` es `NULL` (la clave no está y se devuelve `NULL`), o `r` tiene la clave y le falta un hijo: se devuelve el otro hijo, que puede ser `NULL`, y se libera `r`.
2. Se baja como en `buscar`. Con dos hijos se baja una vez más, al derecho, a borrar el sucesor.
3. Como en `insertar`: lo que devuelve la llamada se guarda en el hijo por el que se bajó, y se devuelve `r`.

`borrar` en C++ (CLRS, §12.3) I

La hoja no tiene rama propia: entra por `r->izq == NULL` y devuelve `r->der`, que también es `NULL`. CLRS mueve el nodo del sucesor con TRANSPLANT; aquí se copia la clave, que deja el mismo conjunto de claves con menos punteros que cambiar.

borrar en C++ (CLRS, §12.3) II

```
// Devuelve la raiz del subarbol sin k. Si k no esta, nada cambia.
Nodo *borrar(Nodo *r, int k) {
  Nodo *res = r;
  if (r != NULL) {
    if (k < r->dato) {
      r->izq = borrar(r->izq, k);
    } else if (k > r->dato) {
      r->der = borrar(r->der, k);
    } else if (r->izq == NULL) {   // hoja o solo hijo derecho
      res = r->der;
      delete r;
    } else if (r->der == NULL) {   // solo hijo izquierdo
      res = r->izq;
      delete r;
    } else {                       // dos hijos: sube el sucesor
      Nodo *suc = minimo(r->der);
      r->dato = suc->dato;
      r->der = borrar(r->der, suc->dato);
    }
  }
  return res;
}
```

`borrar(raiz, 3)`, marco por marco I

Sobre el árbol que quedó tras borrar 13 y 10:

Marco	`r->dato`	Qué hace	Devuelve
1	8	$3 < 8$: `r->izq = borrar(r->izq, 3)`	el nodo del 8
2	3	la tiene y hay dos hijos: `suc` es el 4, `r->dato = 4`, `r->der = borrar(r->der, 4)`	el mismo nodo, ya con 4
3	6	$4 < 6$: `r->izq = borrar(r->izq, 4)`	el nodo del 6
4	4	la tiene y `r->izq` es `NULL`: `res = r->der`, `delete r`	`NULL`

Se baja de arriba abajo y se devuelve de abajo arriba. Al subir, el 6 guarda `NULL` en su izquierdo (el único puntero que cambió de valor además del dato copiado), y los demás marcos reescriben lo que ya tenían.

Casos límite al borrar I

Lo que puede salir mal

- Borrar la raíz. Si tiene a lo sumo un hijo, la raíz cambia; por eso la llamada se escribe `raiz = borrar(raiz, k)`. Sin la asignación, `raiz` queda apuntando a memoria liberada: un puntero colgante.
- Un árbol de un solo nodo: `borrar` devuelve `NULL` y el árbol queda vacío.
- Una clave que no está: la búsqueda llega a `NULL` y nada cambia. `uso_abb.cpp` lo prueba con el 5.
- Con dos hijos se puede usar el predecesor, el máximo del subárbol izquierdo. El árbol que queda es otro ABB válido, con el mismo inorden y otro preorden. Un juez que compara el preorden acepta solo uno de los dos.

La clase ABB I

Las funciones reciben y devuelven raíces. La clase guarda la raíz, hace cada asignación `raiz = ...` en un solo sitio y libera todo en el destructor. Los métodos no se llaman `buscar` ni `insertar`: un método con ese nombre taparía, dentro de la clase, la función libre que llama.

```
// El ABB como objeto: guarda la raiz y libera todo al destruirse.
class ABB {
private:
  Nodo *raiz;

public:
  ABB() {
    raiz = NULL;
  }

  ~ABB() {
    liberar(raiz);
  }
```

La clase ABB II

```
  bool contiene(int k) {
    return buscar(raiz, k) != NULL;
  }

  void agregar(int k) {
    raiz = insertar(raiz, k);
  }

  void quitar(int k) {
    raiz = borrar(raiz, k);
  }

  int alto() {
    return altura(raiz);
  }

  void imprimir() {
    inorden(raiz);
    printf("\n");
  }
};
```

Contenido I

1 Repaso: árbol binario y recursión

2 Buscar rápido sin perder el orden

3 El árbol binario de búsqueda

4 Buscar

5 Insertar

6 Borrar

7 Lo que cuesta: la forma del árbol

8 Ejercicios con recursión

9 Para el juez

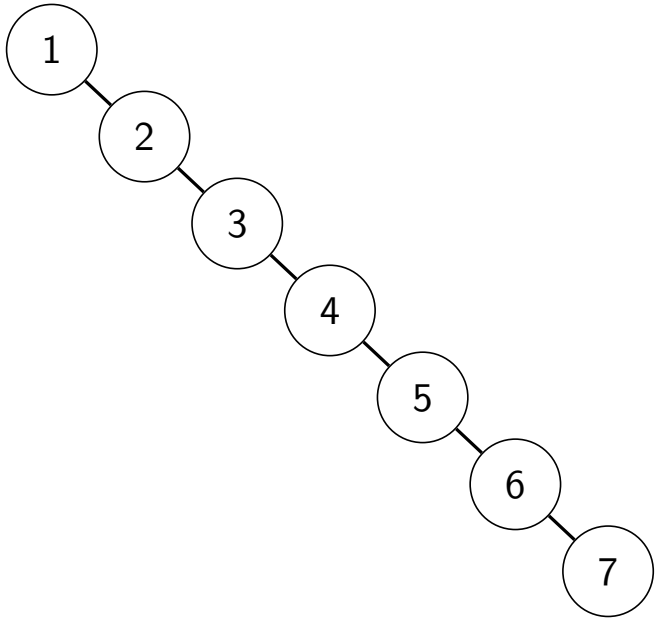
Las mismas siete claves en dos órdenes I

Pregunta

Se insertan las claves del 1 al 7 en un ABB vacío, primero en el orden `1 2 3 4 5 6 7` y después, en otro árbol, en el orden `4 2 6 1 3 5 7`. Dibujen los dos árboles. ¿Qué altura tiene cada uno? ¿Cuántas comparaciones cuesta buscar el 7 en cada uno?

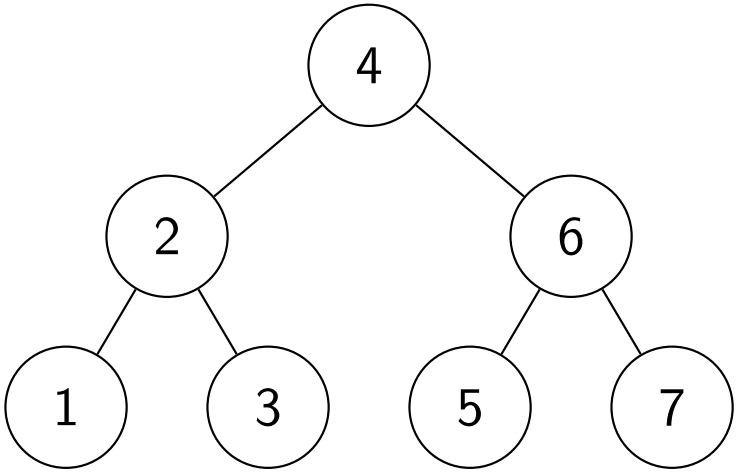
Las mismas siete claves en dos órdenes II

1 2 3 4 5 6 7



altura 6: buscar el 7 compara 7 veces.

4 2 6 1 3 5 7



altura 2: buscar el 7 compara 3 veces.

Las mismas siete claves en dos órdenes III

```
void probar(int claves[], int n) {
  ABB arbol;
  int i = 0;
  while (i < n) {
    printf("%d ", claves[i]);
    arbol.agregar(claves[i]);
    i = i + 1;
  }
  printf("-> altura %d, inorden ", arbol.alto());
  arbol.imprimir();
}

int main() {
  int enOrden[7] = {1, 2, 3, 4, 5, 6, 7};
  int mitades[7] = {4, 2, 6, 1, 3, 5, 7};
  probar(enOrden, 7);
  probar(mitades, 7);
  return 0;
```

Las mismas siete claves en dos órdenes IV

```
1 2 3 4 5 6 7 -> altura 6, inorden 1 2 3 4 5 6 7
4 2 6 1 3 5 7 -> altura 2, inorden 1 2 3 4 5 6 7
```

El inorden es el mismo; la forma no. Insertar en orden creciente siempre baja por la derecha y arma una lista enlazada con un puntero inútil por nodo.

Cuánto puede medir h I

Teorema

Todo árbol binario con $n \geq 1$ nodos tiene altura h con $\lfloor \log_2 n \rfloor \leq h \leq n - 1$.

Demostración. Se procede de forma directa: se cuenta cuántos nodos caben en cada nivel para la cota de abajo, y cuántos nodos tiene un camino para la de arriba.
Desarrollo. El nivel 0 tiene un nodo, la raíz. Cada nodo del nivel $d - 1$ tiene a lo sumo dos hijos, así que el nivel d tiene a lo sumo 2^d nodos. Un árbol de altura h ocupa los niveles 0 a h, entonces

$$n \leq 1 + 2 + \cdots + 2^h = 2^{h+1} - 1 < 2^{h+1}.$$

El logaritmo en base 2 es creciente: $\log_2 n < h + 1$. Como $\lfloor \log_2 n \rfloor \leq \log_2 n$ y los dos lados de $\lfloor \log_2 n \rfloor < h + 1$ son enteros, $\lfloor \log_2 n \rfloor \leq h$.

Cuánto puede medir h II

Para la otra cota: un camino de la raíz a una hoja con h aristas pasa por $h+1$ nodos distintos, y no hay más de n; luego $h+1 \leq n$.

Conclusión. Por lo tanto, se puede concluir que $\lfloor \log_2 n \rfloor \leq h \leq n-1$.

Las dos cotas se alcanzan: 4 2 6 1 3 5 7 da $h = 2 = \lfloor \log_2 7 \rfloor$ y 1 2 3 4 5 6 7 da $h = 6 = n-1$.

El experimento: en orden o desordenadas I

`medir_altura.cpp` inserta n claves en un ABB vacío de dos maneras: `1, 2, ..., n` en orden, y n claves distintas desordenadas con una permutación multiplicativa. Para cada una mide la altura y el tiempo de procesador.

```
// Con P primo mayor que n y 0 < A < P, (i * A) % P da claves
↪  distintas
// para i = 1..n; con A = 623820, cerca de 0,62 P, salen
↪  desordenadas.
const int P = 1000003;
const int A = 623820;

// a[i] = ((i + 1) * A) % P, para i = 0..n-1.
void desordenar(int a[], int n) {
  int i = 0;
  while (i < n) {
    a[i] = (int) ((long long) (i + 1) * A % P);
    i = i + 1;
  }
}
```

El experimento: en orden o desordenadas II

Como P es primo y A no es múltiplo de P, dos valores de i menores que P nunca dan el mismo residuo. Para el ABB solo importa el orden relativo de las claves, no cuáles son.

El experimento: en orden o desordenadas III

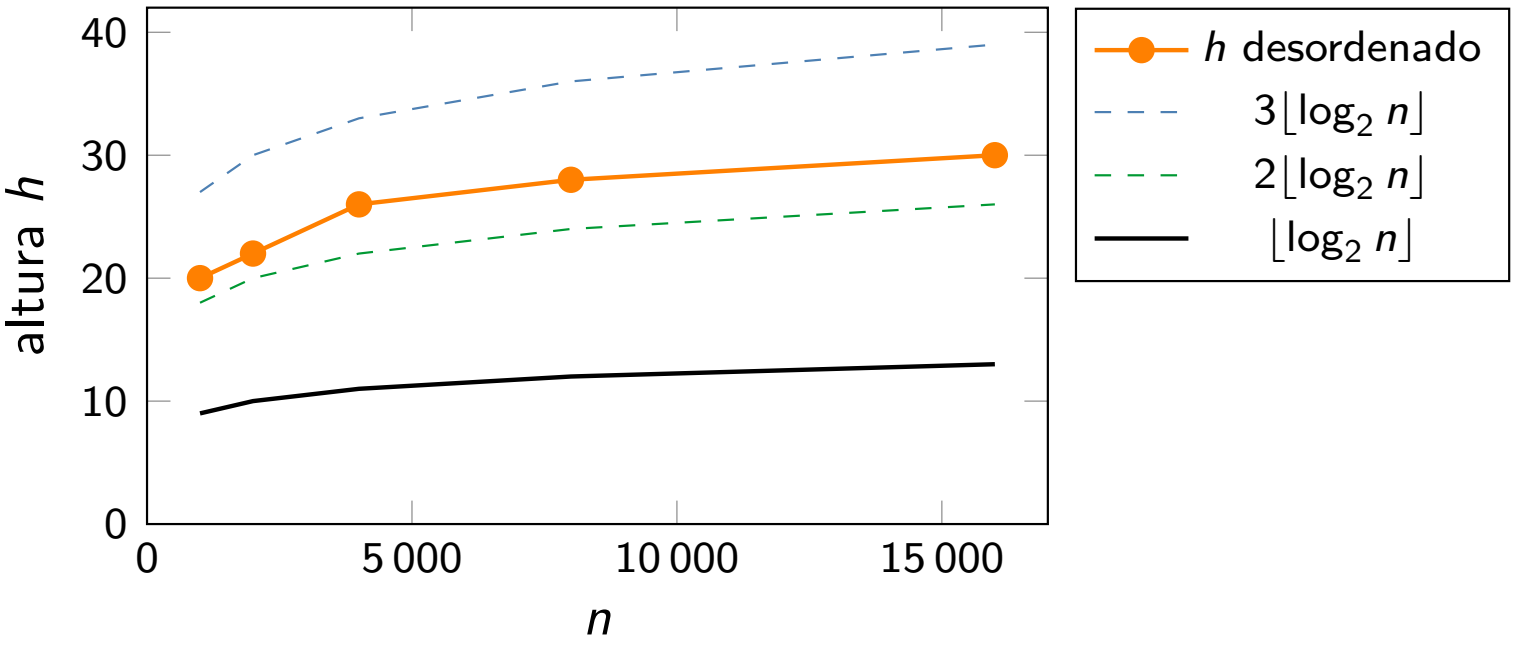
```
void medir(int n) {
  int *a = new int[n];
  int i = 0;
  while (i < n) {
    a[i] = i + 1;
    i = i + 1;
  }
  clock_t t = clock();
  Nodo *orden = construirIterativa(a, n);
  double sOrden = segundos(t);
  desordenar(a, n);
  t = clock();
  Nodo *perm = construirRecursiva(a, n);
  double sPerm = segundos(t);
  printf("%6d %6d %9.4f %6d %9.4f %6d\n", n, altura(orden), sOrden,
         altura(perm), sPerm, pisoLog2(n));
  liberar(orden);
  liberar(perm);
  delete[] a;
}
```

El experimento: en orden o desordenadas IV

n	h en orden	s en orden	h desordenado	s desordenado	$\lfloor \log_2 n \rfloor$	$h/\lfloor \log_2 n \rfloor$
1 000	999	0,0008	20	0,0001	9	2,22
2 000	1 999	0,0032	22	0,0002	10	2,20
4 000	3 999	0,0132	26	0,0004	11	2,36
8 000	7 999	0,0513	28	0,0008	12	2,33
16 000	15 999	0,2543	30	0,0016	13	2,31

- En orden, $h = n - 1$ y el tiempo se multiplica por unos 4 cada vez que n se dobla: las n inserciones cuestan $1 + 2 + \cdots + (n-1) = \Theta(n^2)$ comparaciones.
- Desordenadas, h queda entre 2 y 3 veces $\lfloor \log_2 n \rfloor$ y el tiempo apenas se dobla: el total va como $\Theta(n \log n)$.
- Con 16 000 claves, el árbol en orden es 160 veces más lento.

El experimento: en orden o desordenadas V



La curva en orden no cabe en la gráfica: con $n = 16\,000$ vale $15\,999$.

Lo que cobra el ABB I

Operación	Tiempo	Espacio aparte	Por qué
`buscar`, `insertar`, `borrar`	$O(h)$	$O(h)$	un camino desde la raíz; un marco por nivel
`buscarIterativa`, `insertarIterativa`	$O(h)$	$O(1)$	el mismo camino con uno o dos punteros
`minimo`, `maximo`	$O(h)$	$O(1)$	bajan por un solo lado
`borrar` con dos hijos	$O(h)$	$O(h)$	del nodo al sucesor se sigue bajando: es un solo camino
recorridos, `tamano`, `altura`	$\Theta(n)$	$O(h)$	visitan cada nodo una vez
`liberar`, destructor	$\Theta(n)$	$O(h)$	posorden: los hijos antes que el padre
n inserciones en un árbol vacío	de $\Theta(n \log n)$ a $\Theta(n^2)$	$O(h)$	n caminos, cada uno de largo $O(h)$

Espacio

$\Theta(n)$: un nodo por clave, con la clave y dos punteros.

Lo que cobra el ABB II

Con $\lfloor \log_2 n \rfloor \leq h \leq n - 1$, cada $O(h)$ es $\Theta(\log n)$ cuando el árbol está balanceado y $\Theta(n)$ cuando degeneró en lista. Frente a la tabla del registro de carnés:

	Buscar	Agregar	Quitar	Listar en orden
Arreglo ordenado	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
ABB	$O(h)$	$O(h)$	$O(h)$	$\Theta(n)$

El peor caso llega con los datos más comunes

Carnés que llegan ya ordenados, fechas crecientes, consecutivos de una base de datos: todos arman la lista. Además, la recursión sobre una lista de n nodos apila n marcos: con unos 175 000 nodos desborda la pila de llamadas de 8 MB que da Linux por omisión.

La respuesta a eso son árboles que se reacomodan al insertar y al borrar para que h quede en $O(\log n)$.

Contenido I

1 Repaso: árbol binario y recursión

2 Buscar rápido sin perder el orden

3 El árbol binario de búsqueda

4 Buscar

5 Insertar

6 Borrar

7 Lo que cuesta: la forma del árbol

8 Ejercicios con recursión

9 Para el juez

Ejercicio en parejas: ¿es ABB? I

Se quiere `esABB(r)`, que diga si un árbol binario cualquiera es ABB. Un primer intento revisa cada nodo contra sus hijos:

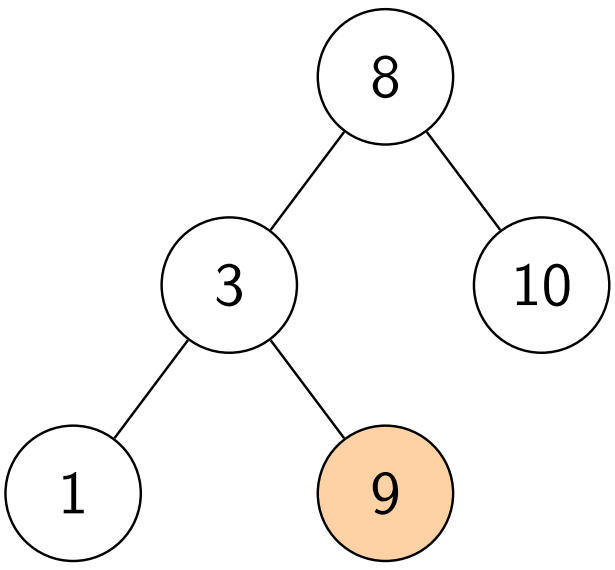
```
// Compara cada nodo solo con sus hijos inmediatos: el arbol falso la
↪  engana.
bool esABBHijos(Nodo *r) {
  bool res = true;
  if (r != NULL) {
    if (r->izq != NULL && r->izq->dato >= r->dato) {
      res = false;
    } else if (r->der != NULL && r->der->dato <= r->dato) {
      res = false;
    } else {
      res = esABBHijos(r->izq) && esABBHijos(r->der);
    }
  }
  return res;
}
```

Ejercicio en parejas: ¿es ABB? II

Pregunta

¿Qué responde `esABBHijos` con el árbol B de la definición, 8; 3 (1, 9); 10? ¿Cuál es el caso base y qué se hace con lo que devuelven las llamadas?

Ejercicio en parejas: ¿es ABB? III



Respuesta

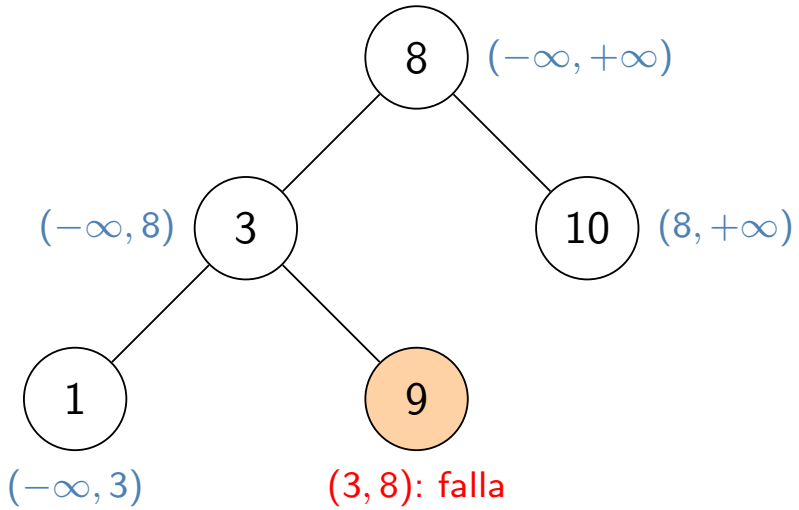
Responde `si`, y se equivoca: el programa imprime `esABBHijos(falso): si`. El caso base es el árbol vacío, que es ABB, y las dos respuestas de abajo se combinan con `&&`. El 9 pasa la revisión porque solo se compara con su padre, el 3. Nadie lo compara con el 8.

Pregunta

Al bajar del 8 al 3, ¿qué se sabe de todas las claves que quedan debajo del 3? ¿Y al bajar del 3 al 9?

esABB con el rango permitido I

Cada nodo hereda un rango abierto $(\text{mín}, \text{máx})$. Al bajar a la izquierda el dato del padre pasa a ser el nuevo máx; a la derecha, el nuevo mín.



esABB con el rango permitido II

```
// Cada clave de r debe quedar estrictamente entre min y max. Al
↪  bajar a
// la izquierda el dato pasa a ser el nuevo max; a la derecha, el
↪  nuevo min.
bool esABB(Nodo *r, long long min, long long max) {
  bool res = true;
  if (r != NULL) {
    res = min < r->dato && r->dato < max
          && esABB(r->izq, min, r->dato) && esABB(r->der, r->dato,
          ↪  max);
  }
  return res;
}
```

La primera llamada usa un rango que deja entrar cualquier `int`:

```
  long long sinMin = (long long) INT_MIN - 1;
  long long sinMax = (long long) INT_MAX + 1;
```

`esABB` con el rango permitido III

- Salida: `esABB(abb): si`, `esABB(falso): no`.
- El `&&` corta en cuanto algo falla: si el dato está fuera de su rango, no se baja a los hijos.
- Cuesta $\Theta(n)$ en tiempo cuando el árbol sí es ABB (revisa cada nodo) y $O(h)$ de pila.

Ejercicio en parejas: el k-ésimo menor I

Pregunta

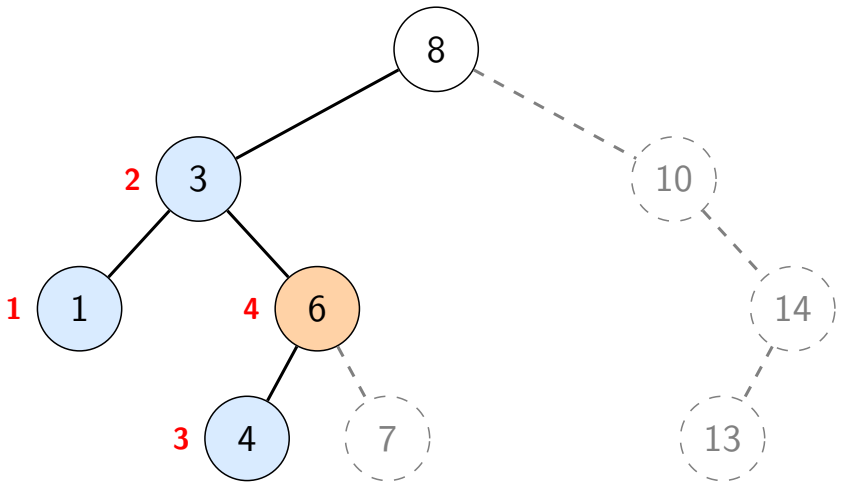
En el árbol de los ejemplos, ¿cuál es la cuarta clave más pequeña? Escriban `kEsimo(r, k, vistos)` sin armar el inorden completo en un arreglo. ¿Qué nodos alcanza a visitar con $k = 4$?

Ejercicio en parejas: el k-ésimo menor II

Es el inorden con un contador `vistos`, pasado por referencia para que todas las llamadas cuenten sobre la misma variable.

```
// k-esima clave menor (k desde 1), o NULL si hay menos de k claves.
// vistos cuenta las claves que el inorden ya visito.
Nodo *kEsimo(Nodo *r, int k, int &vistos) {
  Nodo *res = NULL;
  if (r != NULL) {
    res = kEsimo(r->izq, k, vistos);
    if (res == NULL) {
      vistos = vistos + 1;
      if (vistos == k) {
        res = r;
      } else {
        res = kEsimo(r->der, k, vistos);
      }
    }
  }
  return res;
}
```

Ejercicio en parejas: el k-ésimo menor III



Respuesta

La cuarta es el 6. En rojo, el valor de `vistos` al contar cada nodo. Cuando el 6 llega a 4, `res` deja de ser `NULL` y las llamadas de arriba ya no cuentan ni bajan: el 8 se visitó pero no se contó, y 7, 10, 14 y 13 ni se tocaron.

Salida para $k = 1, \ldots, 10$: `1 3 4 6 7 8 10 13 14 ninguno`.

El k-ésimo con pila explícita I

El mismo recorrido con la pila de `inordenPila`: se cuenta cada nodo al sacarlo de la pila, y la condición del ciclo pide además que `res` siga en `NULL`. Cuando aparece el k-ésimo, el ciclo para sin `break`: la condición del `while` ya no se cumple en la vuelta siguiente.

El k-ésimo con pila explícita II

```
Nodo *kEsimoPila(Nodo *r, int k) {
  Pila p;
  Nodo *res = NULL;
  Nodo *actual = r;
  int vistos = 0;
  while (res == NULL && (actual != NULL || !p.vacia())) {
    if (actual != NULL) {
      p.apilar(actual);
      actual = actual->izq;
    } else {
      actual = p.tope();
      p.desapilar();
      vistos = vistos + 1;
      if (vistos == k) {
        res = actual;
      }
      actual = actual->der;
    }
  }
  return res;
}
```

El k-ésimo con pila explícita III

Vuelta	Hace	Pila (tope a la derecha)	`vistos`
1	apila 8, baja a 3	8	0
2	apila 3, baja a 1	8 3	0
3	apila 1, baja a `NULL`	8 3 1	0
4	saca 1, sigue por su derecho (`NULL`)	8 3	1
5	saca 3, sigue por su derecho (6)	8	2
6	apila 6, baja a 4	8 6	2
7	apila 4, baja a `NULL`	8 6 4	2
8	saca 4, sigue por su derecho (`NULL`)	8 6	3
9	saca 6: `vistos` llega a 4, `res` es el 6	8	4

El ciclo termina con el 8 todavía en la pila; el destructor de `Pila` libera ese nodo. La salida es la misma de `kEsimo`: `1 3 4 6 7 8 10 13 14 ninguno`.

Ejercicio en parejas: contar en un rango I

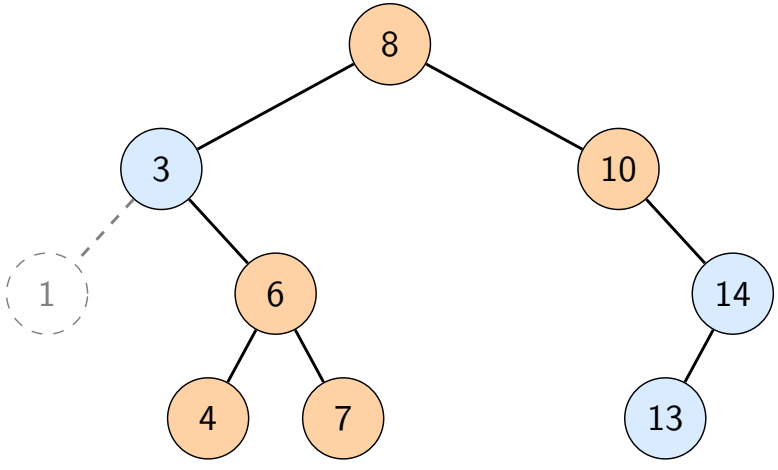
Pregunta

`contarRango(r, a, b)` cuenta las claves en $[a, b]$. En el árbol de los ejemplos, con $[4, 10]$ y con $[11, 12]$: ¿qué subárboles se pueden saltar sin mirarlos? ¿Cuál es el caso base?

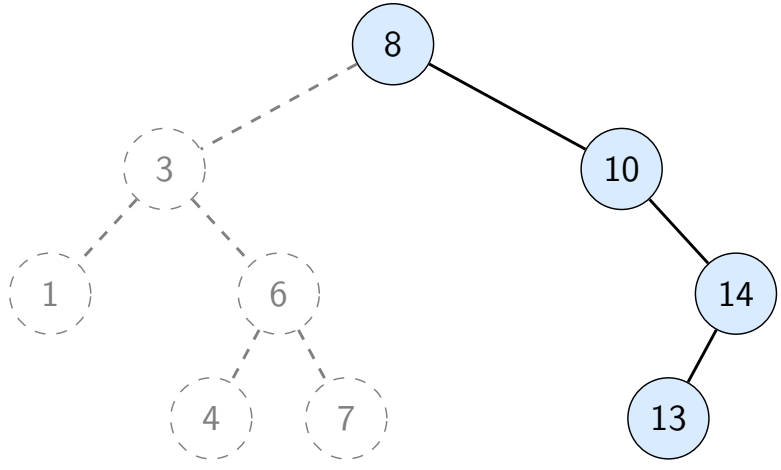
Ejercicio en parejas: contar en un rango II

Si el dato es menor que a, todo su subárbol izquierdo también lo es: solo se baja a la derecha. Si es mayor que b, solo a la izquierda. El caso base es el árbol vacío, que cuenta 0.

$[4, 10]$: cuenta 5



$[11, 12]$: cuenta 0



Naranja: cuenta. Celeste: se visita y no cuenta. Gris: nunca se visita.

Ejercicio en parejas: contar en un rango III

```c
// Claves en [a, b]. Si el dato es menor que a, a la izquierda no hay
// nada que contar; si es mayor que b, a la derecha tampoco.
int contarRango(Nodo *r, int a, int b) {
  int res = 0;
  if (r != NULL) {
    if (r->dato < a) {
      res = contarRango(r->der, a, b);
    } else if (r->dato > b) {
      res = contarRango(r->izq, a, b);
    } else {
      res = 1 + contarRango(r->izq, a, b) + contarRango(r->der, a,
      ↪  b);
    }
  }
  return res;
}
```

- Salida: `contarRango(4, 10)`: 5, `contarRango(11, 12)`: 0, `contarRango(0, 100)`: 9.

Ejercicio en parejas: contar en un rango IV

- Con $[11, 12]$ visita 4 de los 9 nodos. Cuesta $O(h + m)$, con m el número de claves en el rango: se visitan las m y, fuera de ellas, solo nodos de los caminos que buscan a y b.

Ejercicio en parejas: un ABB balanceado desde un arreglo I

Pregunta

Con el arreglo ordenado `1 3 4 6 7 8 10 13 14`, armen el ABB de menor altura posible. ¿Qué clave va en la raíz? Escriban `construirBalanceado(a, ini, fin)` trabajando con los índices, sin copiar mitades del arreglo.

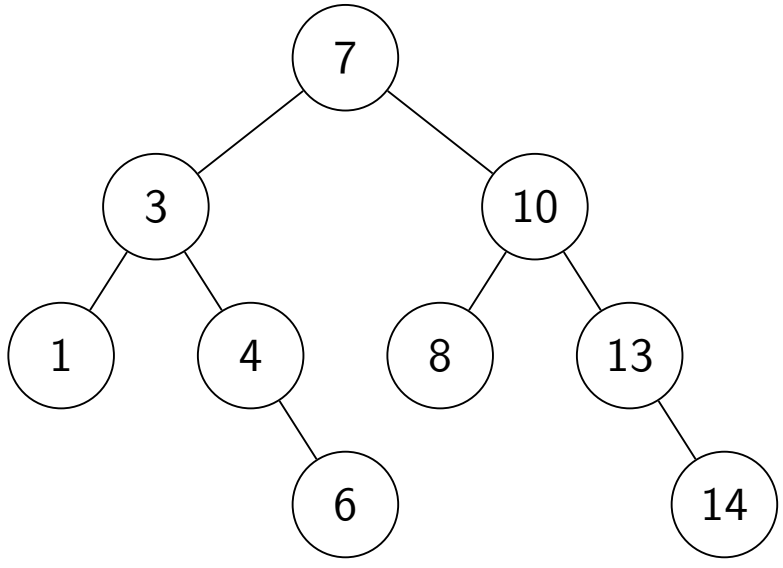
Ejercicio en parejas: un ABB balanceado desde un arreglo II

La raíz es la del medio: deja la mitad de las claves a cada lado. Cada mitad se arma igual, con su propio rango de índices. El rango vacío ($ini > fin$) es el caso base.

```
// ABB de altura minima con a[ini..fin], que viene ordenado:
// el del medio es la raiz y cada mitad, un subarbol.
Nodo *construirBalanceado(int a[], int ini, int fin) {
  Nodo *res = NULL;
  if (ini <= fin) {
    int medio = (ini + fin) / 2;
    res = crearNodo(a[medio], construirBalanceado(a, ini, medio - 1),
                    construirBalanceado(a, medio + 1, fin));
  }
  return res;
}
```

Ejercicio en parejas: un ABB balanceado desde un arreglo III

$[ini, fin]$	`medio`	clave
$[0, 8]$	4	7
$[0, 3]$	1	3
$[0, 0]$	0	1
$[2, 3]$	2	4
$[3, 3]$	3	6
$[5, 8]$	6	10
$[5, 5]$	5	8
$[7, 8]$	7	13
$[8, 8]$	8	14



Salida: `balanceado: preorden 7 3 1 4 6 10 8 13 14 | altura 3, esABB: si`. La altura es $3 = \lfloor \log_2 9 \rfloor$, la mínima posible. Cuesta $\Theta(n)$, una llamada con nodo por clave, y $O(\log n)$ de pila. Insertar el mismo arreglo en orden con `insertar` daría una lista de altura 8.

Ejercicio en parejas: piso y techo I

Pregunta

El piso de x es la mayor clave $\leq x$; el techo, la menor clave $\geq x$. En el árbol de los ejemplos, ¿cuánto valen el piso y el techo de 5, de 12 y de 0? Escriban `piso(r, x)` de forma recursiva.

Ejercicio en parejas: piso y techo II

Si x es menor que el dato, el piso está a la izquierda. Si es mayor, el dato es un candidato: se busca algo mejor a la derecha, y si allá no hay nada, la respuesta es el propio dato.

```
// Nodo con la mayor clave <= x, o NULL si no hay.
Nodo *piso(Nodo *r, int x) {
  Nodo *res = NULL;
  if (r != NULL) {
    if (r->dato == x) {
      res = r;
    } else if (x < r->dato) {
      res = piso(r->izq, x);
    } else {
      res = piso(r->der, x);   // algo mayor que r->dato y <= x
      if (res == NULL) {
        res = r;
      }
    }
  }
  return res;
}
```

Ejercicio en parejas: piso y techo III

`piso(raiz, 12)`, marco por marco

Marco	`r->dato`	Qué hace	Devuelve
1	8	$12 > 8$: busca a la derecha	10 (lo que le llegó)
2	10	$12 > 10$: busca a la derecha	10 (le llegó `NULL`)
3	14	$12 < 14$: busca a la izquierda	`NULL`
4	13	$12 < 13$: busca a la izquierda	`NULL`
5	`NULL`	caso base	`NULL`

Salida del programa: `x = 5: piso 4, techo 6`; `x = 12: piso 10, techo 13`; `x = 0: piso ninguno, techo 1`. `techo` es la misma función con los lados cambiados y queda propuesta.

Lo que cuestan los ejercicios I

Función	Tiempo	Espacio aparte	Por qué
`esABB`	$O(n)$	$O(h)$	visita cada nodo a lo sumo una vez; para en la primera falla
`kEsimo`, `kEsimoPila`	$O(h+k)$	$O(h)$	el inorden hasta la k-ésima, más el camino de bajada; marcos o pila
`contarRango`	$O(h+m)$	$O(h)$	las m claves del rango y los caminos hacia a y b
`construirBalanceado`	$\Theta(n)$	$O(\log n)$	una llamada con nodo por clave; la pila llega a la altura del árbol que arma
`piso`, `techo`	$O(h)$	$O(h)$	un solo camino, como `buscar`

`construirBalanceado` además reserva $\Theta(n)$ para los nodos del árbol que devuelve.

Propuestos para la casa I

1. **Techo.** Escriban `techo(r, x)`, la menor clave $\geq x$, de forma recursiva y después con un ciclo.
2. **Sucesor.** `sucesor(r, k)` devuelve la menor clave estrictamente mayor que `k`, que puede no estar en el árbol. Los nodos no tienen puntero al padre.
3. **Un solo hijo.** Cuenten los nodos que tienen exactamente un hijo. ¿Cuántos tiene el árbol que deja `1 2 3 4 5 6 7`?
4. **De mayor a menor.** Impriman las claves de mayor a menor, primero de forma recursiva y después con una pila explícita.

Propuestos para la casa II

5. **Con el predecesor.** Escriban una versión de `borrar` que, con dos hijos, use el máximo del subárbol izquierdo. Inserten `8 2 3 7 22 1`, borren el 8 con las dos versiones y comparen los preórdenes.
6. **¿Balanceado?** Un árbol está balanceado por alturas si en cada nodo las alturas de sus dos subárboles difieren en a lo sumo 1. Escriban `esBalanceado(r)` llamando a `altura`, y después una versión que calcule la altura en el mismo recorrido.
7. **Orden de inserción.** Impriman un orden de `1, ..., n` que, insertado con `insertar`, deje un ABB de altura $\lfloor \log_2 n \rfloor$, sin crear ningún nodo.

Pistas para los propuestos I

1. Es `piso` con los lados cambiados: si x es mayor que el dato se va a la derecha; si es menor, el dato es candidato y se busca algo mejor a la izquierda. El ciclo guarda el mejor candidato en una variable.
2. Como `techo`, pero el dato es candidato solo si es estrictamente mayor que `k`; si es igual o menor, se va a la derecha. $O(h)$.
3. Caso base: vacío, 0. Se suma 1 si el nodo tiene un hijo y no el otro. El árbol de `1 2 3 4 5 6 7` tiene 6.
4. Es el inorden con los lados cambiados: derecho, raíz, izquierdo. En la versión con pila se baja por la derecha apilando.

Pistas para los propuestos II

5. Con el sucesor sale `22 2 1 3 7`; con el predecesor, `7 2 1 3 22`. El inorden es el mismo, `1 2 3 7 22`.

6. La primera versión llama a `altura` en cada nodo, que recorre todo su subárbol: $O(nh)$, y $\Theta(n \log n)$ si el árbol sí está balanceado. La segunda devuelve la altura, y un valor especial (por ejemplo -2) cuando algo abajo ya falló: $\Theta(n)$.

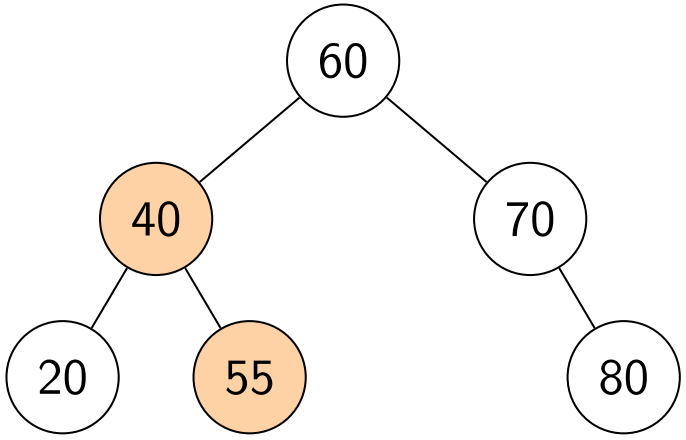
7. Es el preorden de `construirBalanceado`: se imprime el del medio de $[ini, fin]$ y después las dos mitades, sin crear nodos. Cada clave entra después de su padre, y por eso la inserción reconstruye ese mismo árbol.

Ejercicio de cierre I

Pregunta

En un ABB vacío se insertan `50 30 70 20 40 60 80`. Después se borra el 50, se inserta el 55 y se borra el 30, con `borrar` tal como está escrito (sucesor). Dibujen el árbol tras cada paso. ¿Qué preorden y qué altura tiene al final? ¿Qué caso de borrar se usa en cada borrado, incluido el borrado del sucesor?

Ejercicio de cierre II



Respuesta

- Borrar 50: dos hijos. Su sucesor es el 60, una hoja (caso 1 abajo). La raíz pasa a ser 60 y el 70 queda sin hijo izquierdo.
- Insertar 55: $55 < 60$, $55 > 30$, $55 > 40$; entra como hijo derecho del 40.
- Borrar 30: dos hijos. Su sucesor es el 40, que no tiene hijo izquierdo pero sí derecho: abajo es el caso 2 y el 55 sube.
- Preorden 60 40 20 55 70 80, altura 2.

Contenido I

1 Repaso: árbol binario y recursión

2 Buscar rápido sin perder el orden

3 El árbol binario de búsqueda

4 Buscar

5 Insertar

6 Borrar

7 Lo que cuesta: la forma del árbol

8 Ejercicios con recursión

9 Para el juez

Para el juez: Aizu ALDS1_8_A I

Aizu ALDS1_8_A, Binary Search Tree I (Aizu Online Judge)
Envío: `https://vjudge.net/problem/Aizu-ALDS1_8_A`
Enunciado:
`https://onlinejudge.u-aizu.ac.jp/problems/ALDS1_8_A`

Qué pide

Llegan $m \leq 500\,000$ órdenes, `insert k` o `print`. Cada `print` imprime dos líneas: el inorden y el preorden del ABB. Las claves son distintas, entre $-2 \cdot 10^9$ y $2 \cdot 10^9$; hay a lo sumo 10 `print`, y el enunciado garantiza que la altura no pasa de 100.

Para el juez: Aizu ALDS1_8_A II

¿Entendimos el problema?

Con `insert` de 30, 88, 12, 1, 20, 17 y 25 y un `print`, la salida es `1 12 17 20 25 30 88` y `30 12 1 20 17 25 88`. ¿Por qué el preorden empieza en 30 y no en 1?

Porque el preorden empieza por la raíz, y la raíz es la primera clave insertada.

Cómo atacarlo

`insertar` de la sesión para cada `insert`, y los recorridos para `print`. Con altura a lo sumo 100, cada inserción hace unas 100 comparaciones y la recursión apila pocos marcos. En total, $O(m \cdot h)$ para las inserciones y $\Theta(n)$ por `print`; espacio $\Theta(n)$.

Para el juez: Aizu ALDS1_8_A III

Las trampas

- Cada clave va precedida de un espacio, y la línea no termina en espacio. El `inorden` de `arbol_binario.h` imprime cada clave seguida de un espacio: hay que escribir otro que la imprima precedida de uno, con `printf(" %d", r->dato)`.
- Las claves caben en `int`, pero quedan cerca del borde ($2^{31} - 1 \approx 2{,}1 \cdot 10^9$).
- La orden se lee con `scanf("%s", orden)` y se compara con `strcmp`.
- La garantía de altura es lo que vuelve viable un ABB sin balancear con medio millón de órdenes.

Para el juez: Aizu ALDS1_8_B I

Aizu ALDS1_8_B, Binary Search Tree II (Aizu Online Judge)
Envío: `https://vjudge.net/problem/Aizu-ALDS1_8_B`
Enunciado:
`https://onlinejudge.u-aizu.ac.jp/problems/ALDS1_8_B`

Qué pide

Lo mismo que ALDS1_8_A más la orden `find k`, que imprime `yes` o `no`. Los límites no cambian: $m \leq 500\,000$ y altura a lo sumo 100.

¿Entendimos el problema?

En el ejemplo, `find 12` llega después de insertar 30, 88, 12, 1 y 20, y responde `yes`; `find 16` responde `no`. ¿En qué nodo se cae la búsqueda del 16?

En el hijo izquierdo del 17, que es `NULL`: $16 < 30$, $16 > 12$, $16 < 20$, $16 < 17$.

Para el juez: Aizu ALDS1_8_B II

Cómo atacarlo

La solución de ALDS1_8_A con `buscar` para `find`. Con la versión iterativa, cada `find` cuesta $O(h)$ en tiempo y $O(1)$ en espacio.

La trampa

No trae una nueva. Las respuestas de `find` se imprimen en el momento, mezcladas con las líneas de `print`, en el orden en que llegan las órdenes.

Para el juez: UVa 12347 I

UVa 12347, Binary Search Tree (UVa Online Judge)
Envío: `https://vjudge.net/problem/UVA-12347`
Enunciado: `https://vjudge.net/problem/UVA-12347`

Qué pide

Llega el preorden de un ABB, una clave por línea, hasta el fin de la entrada. Las claves son enteros positivos distintos, menores que 10^6, y hay a lo sumo 10 000. Se imprime el posorden, una clave por línea. El límite de tiempo es de 1 segundo.

Para el juez: UVa 12347 II

¿Entendimos el problema?

La entrada 50 30 24 5 28 45 98 52 60 da la salida 5 28 24 45 30 60 52 98 50. ¿Por qué insertar las claves en el orden en que llegan reconstruye el mismo árbol?

En el preorden cada clave aparece después de todos sus antepasados. Al insertarla, la búsqueda recorre esos antepasados y la cuelga en el mismo lugar que tenía.

Cómo atacarlo

`insertar` con cada clave leída y `posorden` al final. Cuesta $O(n \cdot h)$: con un árbol balanceado, $\Theta(n \log n)$; con uno degenerado, unas $n^2/2 = 5 \cdot 10^7$ comparaciones, que caben en el segundo.

Para el juez: UVa 12347 III

Las trampas

- No se da la cantidad de claves: se lee con `while (scanf("%d", &x) == 1)`.
- Un preorden ordenado arma una lista de altura 9 999. Es el ejemplo concreto de que el costo es $O(h)$ y no $O(\log n)$.
- En esa lista, `insertar` y `posorden` apilan 10 000 marcos. Caben en la pila, pero `insertarIterativa` se ahorra la mitad del problema.

Para el juez: Aizu ALDS1_8_C I

Aizu ALDS1_8_C, Binary Search Tree III (Aizu Online Judge)
Envío: `https://vjudge.net/problem/Aizu-ALDS1_8_C`
Enunciado:
`https://onlinejudge.u-aizu.ac.jp/problems/ALDS1_8_C`

Qué pide

Lo de ALDS1_8_B más `delete k`, que borra la clave con los tres casos y no imprime nada. Mismos límites: $m \leq 500\,000$ y altura a lo sumo 100.

¿Entendimos el problema?

Se insertan 8, 2, 3, 7, 22 y 1, y luego se borran el 3 y el 7. El último `print` da `1 2 8 22` y `8 2 1 22`. ¿Qué caso de borrar es cada uno?

El 3 tiene un solo hijo, el 7, que sube a su lugar (caso 2). Después el 7 es una hoja (caso 1).

Para el juez: Aizu ALDS1_8_C II

Cómo atacarlo

La solución de ALDS1_8_B con `borrar` de la sesión, que devuelve la nueva raíz del subárbol: `raiz = borrar(raiz, k)`. Cada orden cuesta $O(h)$.

Las trampas

- Con dos hijos hay que subir el sucesor, el mínimo del subárbol derecho, como pide el enunciado. El ejemplo no tiene ese caso. Uno propio: insertar 8, 2, 3, 7, 22 y 1, y borrar el 8. Con el sucesor, el preorden es `22 2 1 3 7`; con el predecesor, `7 2 1 3 22`. El inorden coincide y el juez marca respuesta incorrecta.
- Al borrar la raíz, la raíz cambia: sin la asignación `raiz = ...`, el árbol queda colgando de un nodo liberado.
- El enunciado no dice qué pasa con una clave ausente; la solución no debe fallar en ese caso.

Reto: UVa 10821 I

UVa 10821, Constructing BST (UVa Online Judge)
Envío: `https://vjudge.net/problem/UVA-10821`
Enunciado: `https://vjudge.net/problem/UVA-10821`

Qué pide

Para cada par N, H, con $1 \leq N \leq 10\,000$ y $1 \leq H \leq 30$, el orden de inserción de $1, \ldots, N$ que deja un ABB de altura a lo sumo H y que es el menor en orden lexicográfico. La entrada termina con `0 0`. Si no existe, se imprime `Impossible.`

¿Entendimos el problema?

`4 3` da `Case 1: 1 3 2 4`; `4 1` da `Case 2: Impossible.`; `6 3` da `Case 3: 3 1 2 5 4 6`. ¿Por qué con 4 claves y $H = 3$ la raíz puede ser el 1?

Reto: UVa 10821 II

En este problema un nodo solo tiene altura 1, así que con $H = 3$ cada subárbol de la raíz tiene altura 2 y caben 3 claves. Con raíz 1, a la derecha quedan 2, 3 y 4: caben.

Cómo atacarlo

Recursión sobre un rango $[ini, fin]$ con la altura que le queda, como en `construirBalanceado`. En un árbol de altura H caben $2^H - 1$ claves. La raíz es la menor clave que deja al subárbol derecho caber en altura $H - 1$; se imprime y se repite con las dos mitades. El orden de impresión es un preorden. Cuesta $\Theta(N)$ por caso.

Reto: UVa 10821 III

Las trampas

- Aquí el árbol vacío tiene altura 0 y una hoja, altura 1: uno más que en la sesión.
- $2^{30} - 1$ cabe en `int`; 2^{31} no.
- `Impossible.` lleva punto, y la línea no termina en espacio.

Resumen I

- En un ABB, para cada nodo, el subárbol izquierdo tiene claves menores y el derecho, mayores. Revisar solo los hijos inmediatos no basta; hace falta el rango permitido.
- El inorden de un ABB sale ordenado.
- Buscar, insertar y borrar bajan por un solo lado. Insertar y borrar devuelven la raíz del subárbol, y quien llama la guarda en el hijo por el que bajó: así se reengancha el árbol sin preguntar por el padre.
- Borrar tiene tres casos: hoja, un hijo (sube el hijo) y dos hijos (sube la clave del sucesor y el sucesor se borra abajo).
- Todas cuestan $O(h)$; las recursivas piden $O(h)$ de pila y las iterativas $O(1)$. Como $\lfloor \log_2 n \rfloor \leq h \leq n - 1$, el mismo árbol va de $\Theta(\log n)$ a $\Theta(n)$ según el orden de inserción.

Resumen II

Lo que sigue

El contenedor `set`, que guarda las claves en un árbol binario de búsqueda que la biblioteca mantiene balanceado: buscar, insertar y borrar cuestan $O(\log n)$ siempre, y recorrerlo da las claves en orden. Después, los montículos: un árbol binario completo guardado en un arreglo, donde la clave mayor siempre está en la raíz.

Referencias I



T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Apéndice B.5, árboles; §10.3, representación de árboles con raíz; capítulo 12, árboles binarios de búsqueda (§12.1, definición y recorridos; §12.2, buscar, mínimo, máximo y sucesor; §12.3, insertar y borrar).



R. Thareja. *Data Structures Using C*, 2.ª ed. Oxford University Press, 2014. Capítulo 9, *Trees*; capítulo 10, *Efficient Binary Trees*: §10.1, árboles binarios de búsqueda, y §10.2, sus operaciones.



Aizu Online Judge. Problemas ALDS1_8_A, ALDS1_8_B y ALDS1_8_C, *Binary Search Tree I, II, III*: `https://onlinejudge.u-aizu.ac.jp/problems/ALDS1_8_A`, y lo mismo con `ALDS1_8_B` y `ALDS1_8_C`.



UVa Online Judge. Problema 12347, *Binary Search Tree*: `https://vjudge.net/problem/UVA-12347`.



UVa Online Judge. Problema 10821, *Constructing BST*: `https://vjudge.net/problem/UVA-10821`.